Интеграция обучения с подкреплением и планирования. Модели мира

Алексей Скрынник

ПЛАН

01 Монте-Карло поиск по дереву

02 AlphaZero

03 MuZero

04 Применения и улучшения

05 Dreamer

01

Монте-Карло поиск по дереву

Полный поиск: минимакс

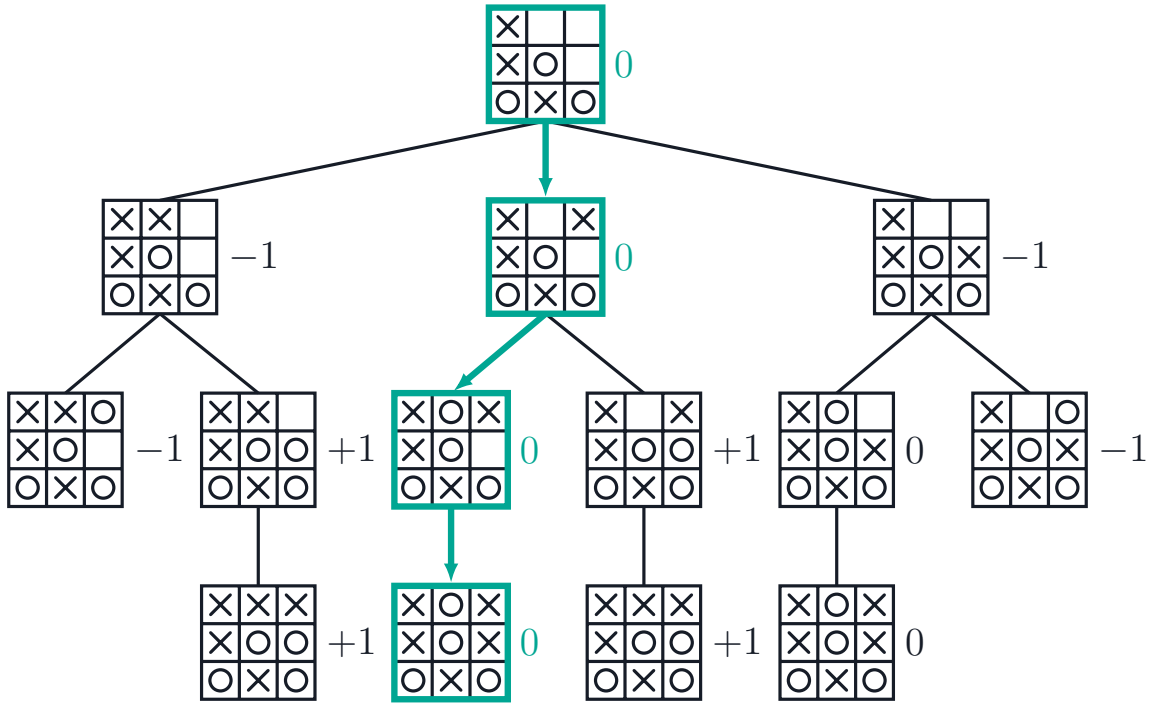
Перебор до конца игры, снизу вверх:

ходят крестики: $V(s) = \max\limits_{s'} V(s')$

ходят нолики: $V(s) = \min\limits_{s'} V(s')$

s' — позиции после одного хода из s;

в конце игры $V = +1$, 0 или -1 для крестиков.



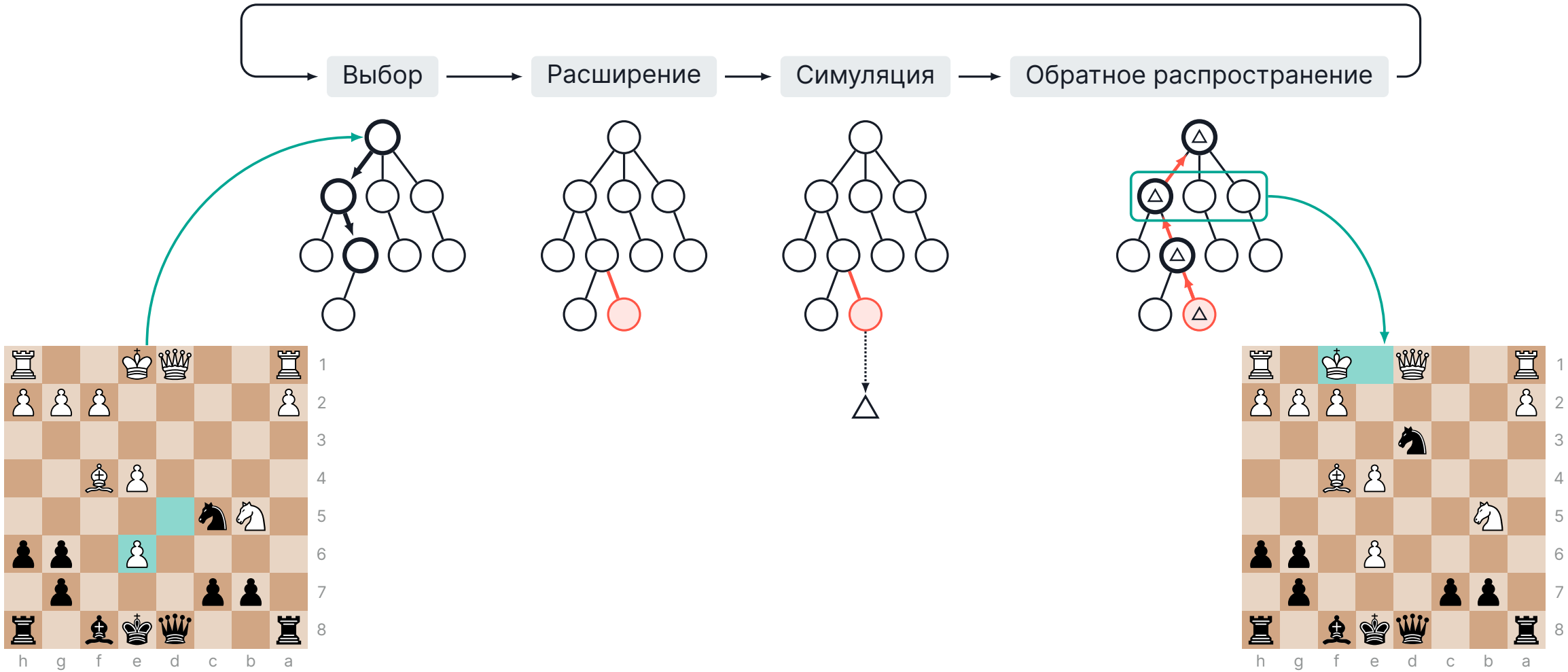
Размер дерева $\approx b^d$:

b — ходов в позиции, d — длина партии.

Шахматы	$b \approx 35$	$d \approx 80$	$\Rightarrow$	10^{123}
Го	$b \approx 250$	$d \approx 150$	$\Rightarrow$	10^{360}

Полный перебор невозможен $\Rightarrow$ MCTS строит дерево **выборочно**.

Монте-Карло поиск по дереву (MCTS)

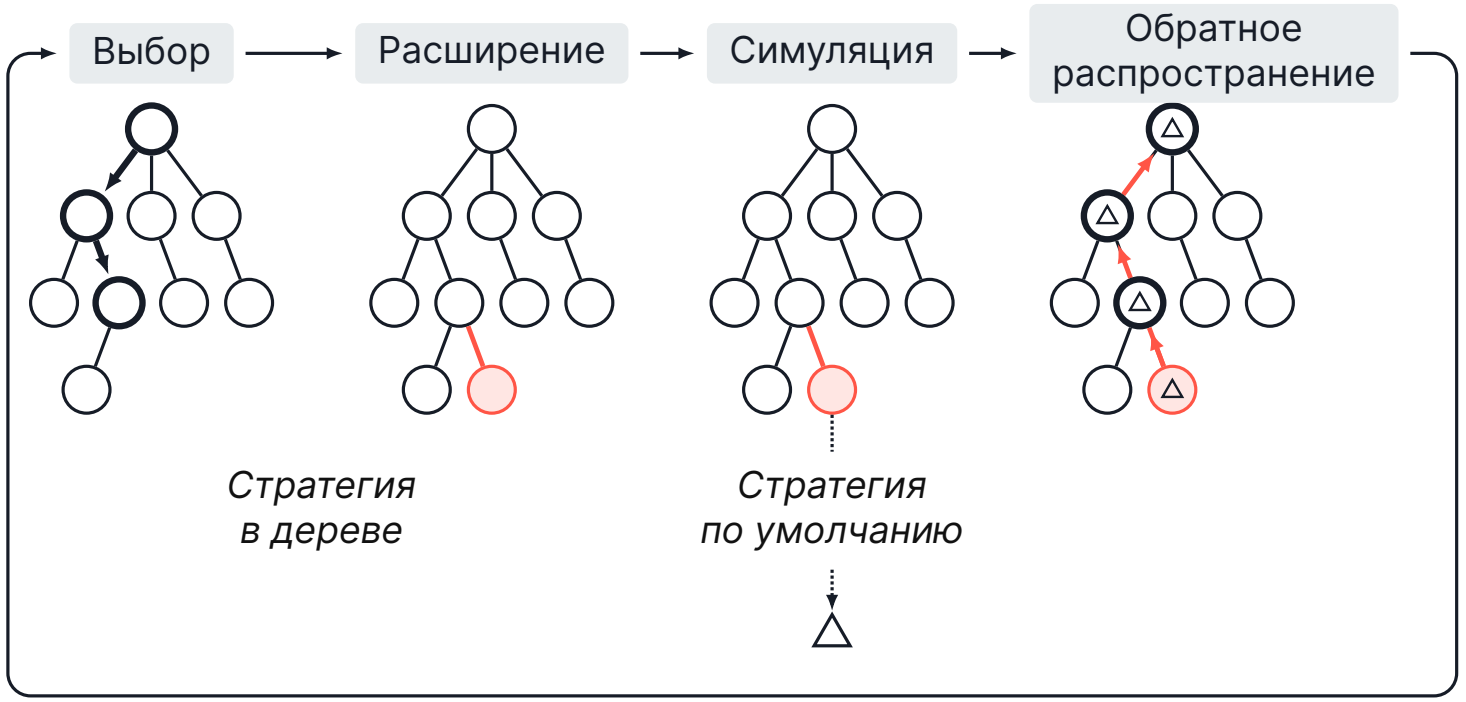


Монте-Карло поиск по дереву (MCTS)

Как выбрать одно действие?

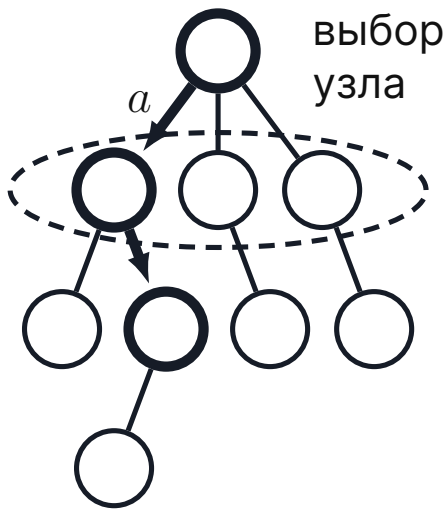
Алгоритм 1. Общая схема MCTS

function MCTSSEARCH(s_t)
  создать корень для s_t
  while бюджет не исчерпан **do**
    $s_L \leftarrow$ TREEPOLICY(s_t)
    $G \leftarrow$ DEFAULTPOLICY(s_L)
    BACKUP(s_L, G)
  return BESTCHILD(s_t)



Browne et al. *A Survey of Monte Carlo Tree Search Methods*. IEEE Trans. Comput. Intell. AI Games, 2012. doi:10.1109/TCIAIG.2012.2186810.

MCTS: выбор



выбор следующего узла

$$\mathrm{UCT}(s,a) = \frac{W(s,a)}{N(s,a)} + c \cdot \sqrt{\frac{\overbrace{\ln N(s)}}{N(s,a)}}$$

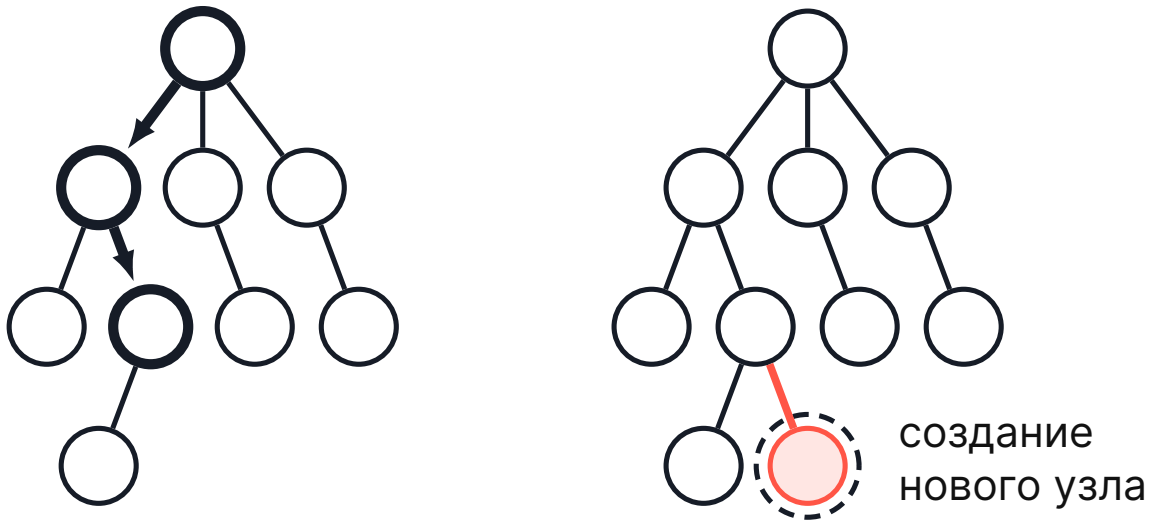
средний результат

число посещений узла s

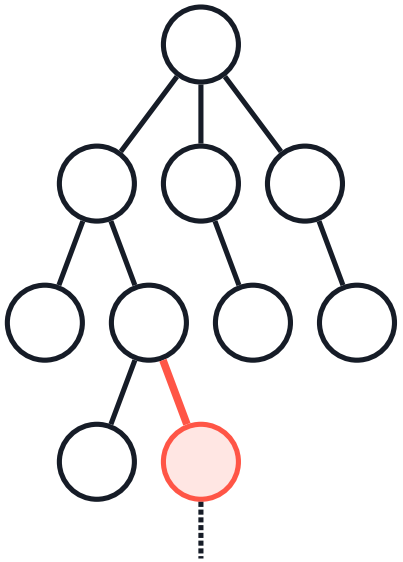
число посещений ребра (s,a)

Статистики $N(s,a)$, $W(s,a)$ хранятся на рёбрах дерева; $N(s) = \sum_b N(s,b)$. Непосещённые действия ($N(s,a) = 0$) выбираются первыми.

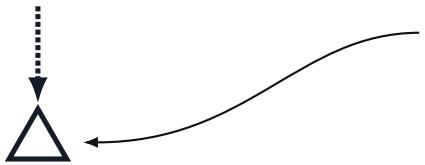
MCTS: расширение



MCTS: симуляция

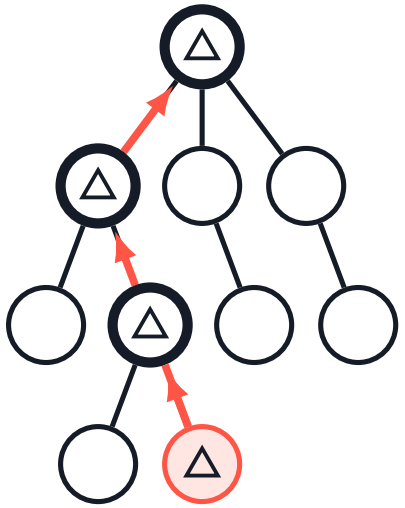


Розыгрыш
по стратегии
(например, случайной)



результат G: победа / поражение

MCTS: обратное распространение



$$N(s,a) \leftarrow N(s,a) + 1$$
$$W(s,a) \leftarrow W(s,a) + G$$
$$Q(s,a) = W(s,a) \,/\, N(s,a)$$

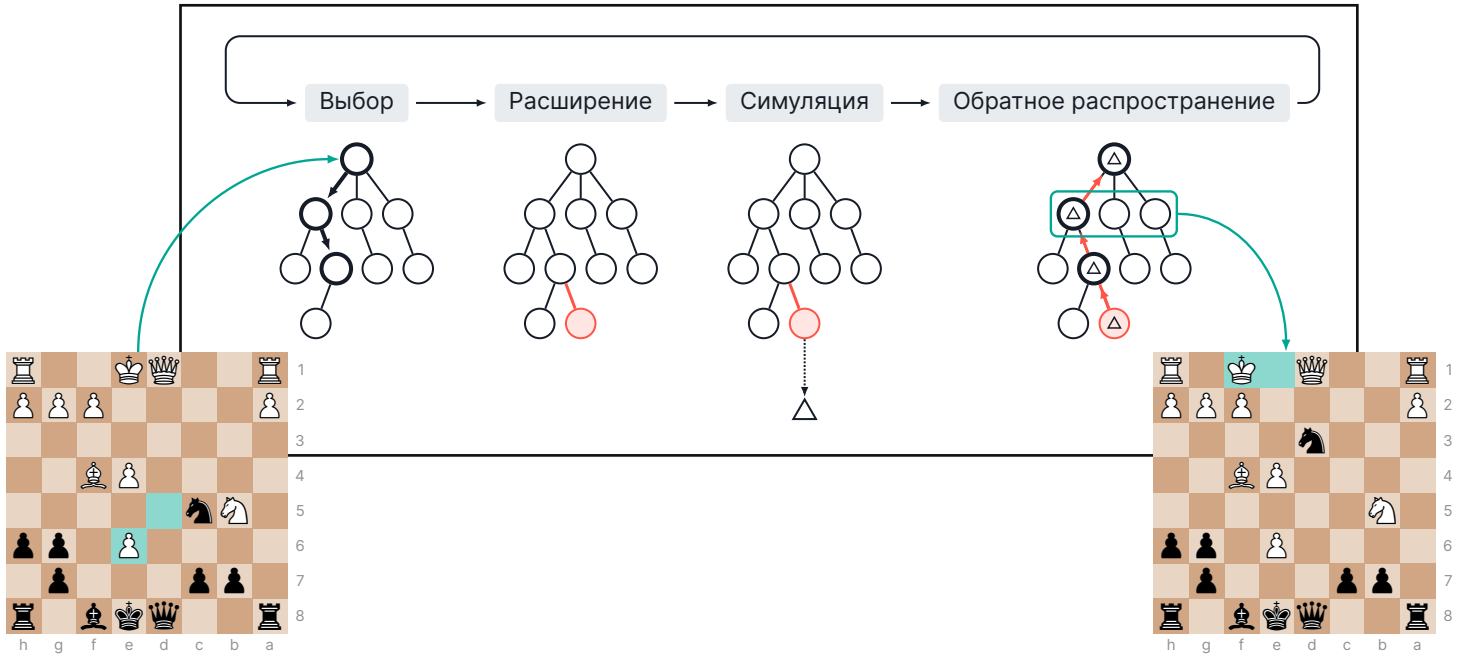
G — результат симуляции: $+1$ (победа) или -1 (поражение) для игрока, который сделал ход a в s; поэтому знак G чередуется по уровням дерева.

Монте-Карло поиск по дереву (MCTS)

Алгоритм 1 запускается на каждом шаге t эпизода из состояния s_t

Алгоритм 1. Общая схема MCTS

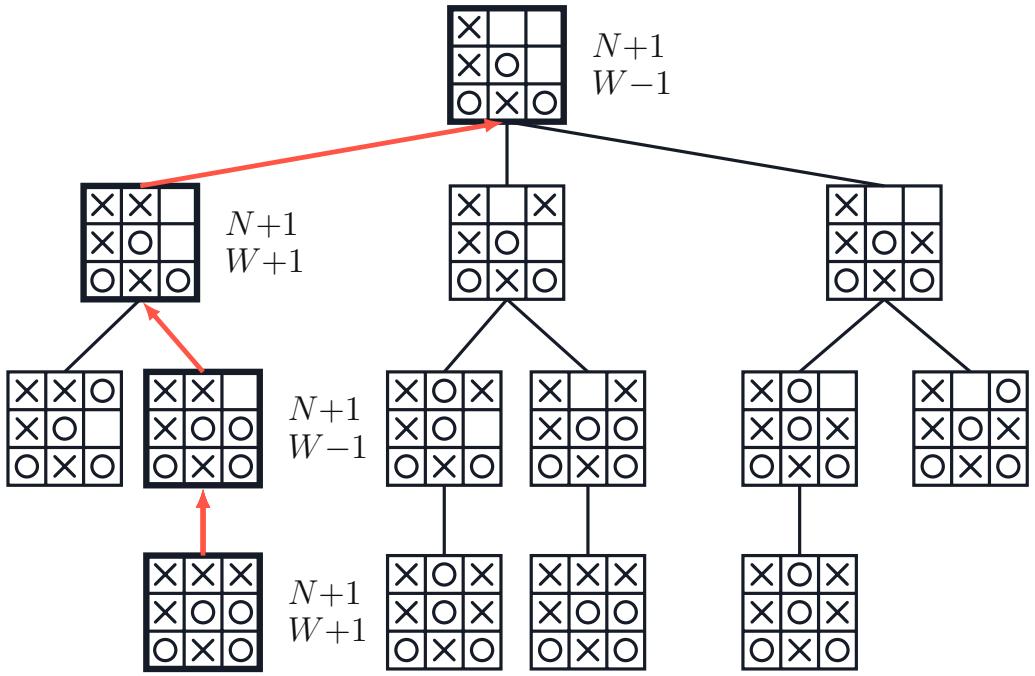
function MCTSSEARCH(s_t)
  создать корень для s_t
  while бюджет не исчерпан **do**
    $s_L \leftarrow$ TREEPOLICY(s_t)
    $G \leftarrow$ DEFAULTPOLICY(s_L)
    BACKUP(s_L, G)
  return BESTCHILD(s_t)



Browne et al. *A Survey of Monte Carlo Tree Search Methods*. IEEE Trans. Comput. Intell. AI Games, 2012. doi:10.1109/TCIAIG.2012.2186810.

Классический MCTS: итоги

- Умная альтернатива полному перебору
- MCTS применяли в планировании, составлении расписаний и физическом моделировании (например, алгоритм HOOT)
- Наибольшую известность получил благодаря игре го
- Может показывать высокое качество при хорошей стратегии по умолчанию (для небольшого пространства действий)

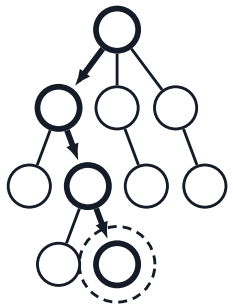


02

AlphaZero

AlphaZero: этапы построения дерева

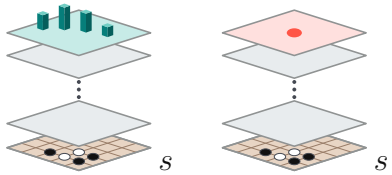
1 Как выбирать действия в дереве?



Q + U

Сеть стратегии $\pi_\theta(a \mid s)$

Сеть полезности $V_\phi(s)$



Средняя полезность следующего состояния

Функция от $P(s,a)$ и $N(s,a)$: растёт, если действие мало исследовано по сравнению с другими или если его априорная вероятность высока

среднее оценок критика V_ϕ

исследование

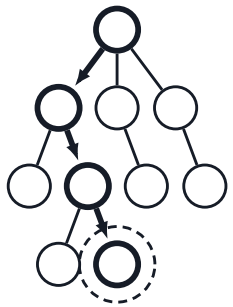
$$\text{pUCT}(s,a) = Q(s,a) + cP(s,a)\frac{\sqrt{N(s)}}{1+N(s,a)}$$

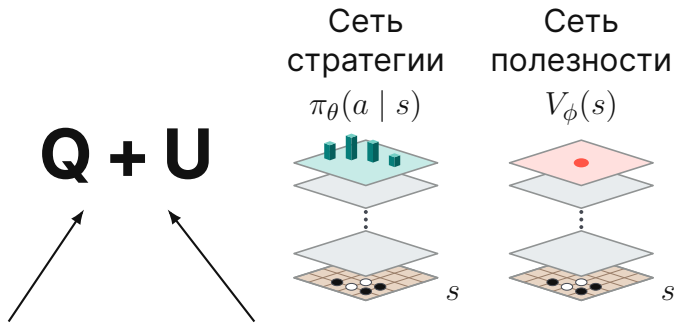
Априорная вероятность от актора: $P(s,a) = \pi_\theta(a \mid s)$

pUCT: Schrittwieser et al. *Mastering Atari, Go, Chess and Shogi by Planning with a Learned Model*. Nature, 2020. arXiv:1911.08265.
Сети стратегии и полезности: Silver et al. *Mastering the game of Go with deep neural networks and tree search*. Nature, 2016. doi:10.1038/nature16961.

AlphaZero: этапы построения дерева

1 Как выбирать действия в дереве?





Средняя полезность следующего состояния

Функция от $P(s,a)$ и $N(s,a)$: растёт, если действие мало исследовано по сравнению с другими или если его априорная вероятность высока

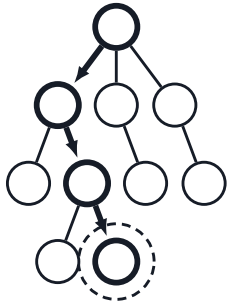
среднее оценок критика V_ϕ

исследование

$$\mathrm{pUCT}(s,a) = Q(s,a) + cP(s,a)\frac{\sqrt{N(s)}}{1+N(s,a)}$$

Априорная вероятность от актора:
$P(s,a) = \pi_\theta(a \mid s)$

2 Продолжаем до неисследованного узла



Состояние листа s_L подаётся в сети стратегии и полезности, которые предсказывают две величины:

$P(s_L, \cdot) = \pi_\theta(\cdot \mid s_L)$ — вероятности ходов

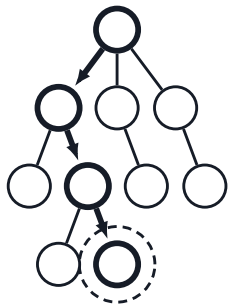
$V_\phi(s_L)$ — полезность состояния (для текущего игрока)

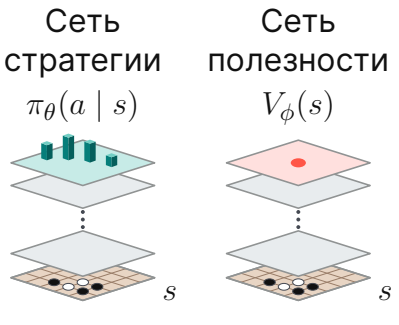
Вероятности $P(s_L, a)$ приписываются новым допустимым действиям a из листа

pUCT: Schrittwieser et al. *Mastering Atari, Go, Chess and Shogi by Planning with a Learned Model*. Nature, 2020. arXiv:1911.08265.
Сети стратегии и полезности: Silver et al. *Mastering the game of Go with deep neural networks and tree search*. Nature, 2016. doi:10.1038/nature16961.

AlphaZero: этапы построения дерева

1 Как выбирать действия в дереве?





Средняя полезность следующего состояния

Функция от $P(s,a)$ и $N(s,a)$: растёт, если действие мало исследовано по сравнению с другими или если его априорная вероятность высока

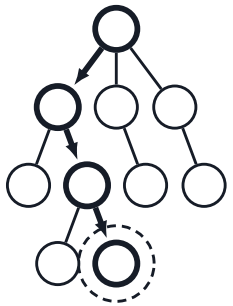
среднее оценок критика V_ϕ

исследование

$$\text{pUCT}(s,a) = Q(s,a) + c\,P(s,a)\,\frac{\sqrt{N(s)}}{1+N(s,a)}$$

Априорная вероятность от актора:
$P(s,a) = \pi_\theta(a \mid s)$

2 Продолжаем до неисследованного узла



Состояние листа s_L подаётся в сети стратегии и полезности, которые предсказывают две величины:

$P(s_L, \cdot) = \pi_\theta(\cdot \mid s_L)$ — вероятности ходов

$V_\phi(s_L)$ — полезность состояния (для текущего игрока)

Вероятности $P(s_L, a)$ приписываются новым допустимым действиям a из листа

3 Обратное распространение

Каждое ребро на пути к листу обновляется так:

$$N(s,a) \leftarrow N(s,a) + 1$$

$$W(s,a) \leftarrow W(s,a) + G$$

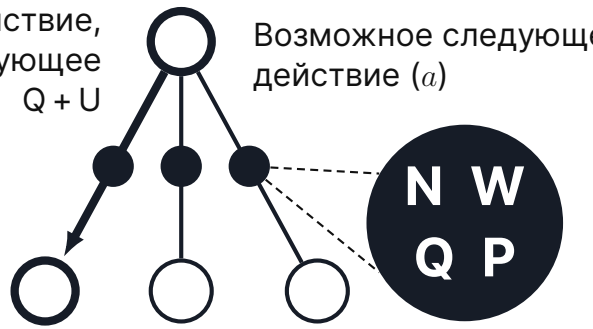
$$Q(s,a) = W(s,a) \,/\, N(s,a)$$

$G = V_\phi(s_L)$ — оценка критика вместо симуляции

Текущее состояние игры (s)

Действие, максимизирующее Q + U

Возможное следующее действие (a)



pUCT: Schrittwieser et al. *Mastering Atari, Go, Chess and Shogi by Planning with a Learned Model*. Nature, 2020. arXiv:1911.08265.
Сети стратегии и полезности: Silver et al. *Mastering the game of Go with deep neural networks and tree search*. Nature, 2016. doi:10.1038/nature16961.

Как выбрать действие после построения дерева?

После 1600 симуляций ход выбирается одним из двух способов.

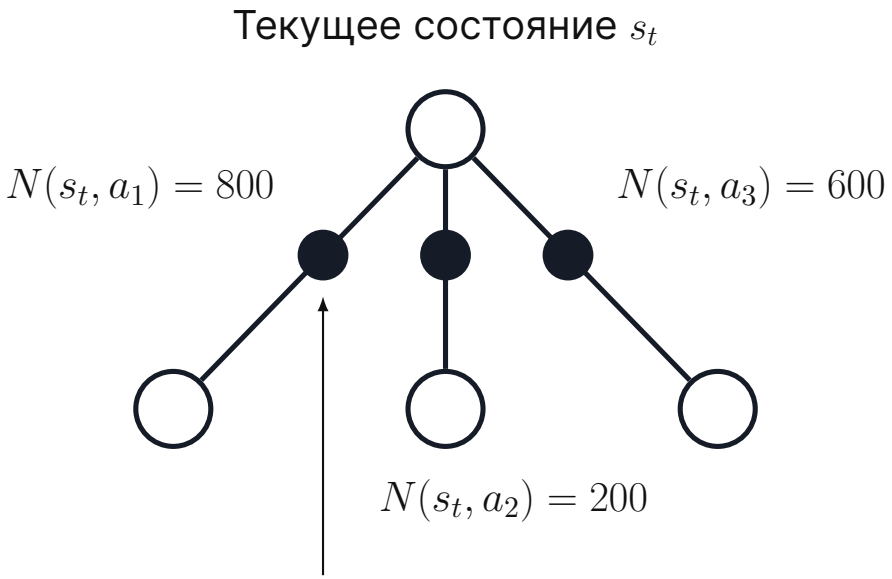
а) Наиболее исследованное поддерево
детерминированно, для соревновательной игры:

$$a_t = \arg\max_a N(s_t, a)$$

б) Сэмплирование из распределения
стохастически, для исследования:

$$a_t \sim \pi_{\text{MCTS}}(\cdot \mid s_t), \qquad \pi_{\text{MCTS}}(a \mid s_t) \propto N(s_t, a)^{1/\tau}$$

τ — температура, управляет исследованием.



Детерминированно: выбираем этот ход

Стохастически: сэмплируем ход из
$\pi_{\text{MCTS}}(\cdot \mid s_t) = (0.5,\ 0.125,\ 0.375)$
при $\tau = 1$

Поиск по дереву — оператор улучшения стратегии:
после серии симуляций стратегия π_{MCTS} сильнее исходной π_θ.

По мотивам: D. Foster. *AlphaGo Zero Explained In One Diagram*. Medium, 2017.

Как обучается AlphaZero?

1 Игра с собой

Создаём обучающую выборку

Игра против своей копии $\Rightarrow$
неявный curriculum learning

На каждом ходе сохраняются:



Состояние s_t

π

Вероятности поиска $\pi_{\text{MCTS}}(\cdot \mid s_t)$



Победитель $z \in \{-1, 0, +1\}$ (после партии)

улучшенная поиском стратегия:
$\pi_{\text{MCTS}}(a \mid s_t) \propto N(s_t, a)^{1/\tau}$

По мотивам: D. Foster. *AlphaGo Zero Explained In One Diagram*. Medium, 2017; Silver et al. *Mastering the game of Go without human knowledge*. Nature, 2017. doi:10.1038/nature24270.

Как обучается AlphaZero?

1 Игра с собой

Создаём обучающую выборку

Игра против своей копии ⇒ неявный curriculum learning

На каждом ходе сохраняются:



Состояние s_t

π

Вероятности поиска $\pi_{\text{MCTS}}(\cdot \mid s_t)$



Победитель $z \in \{-1, 0, +1\}$ (после партии)

улучшенная поиском стратегия:

$\pi_{\text{MCTS}}(a \mid s_t) \propto N(s_t, a)^{1/\tau}$

2 Дообучение сети

Оптимизируем веса сети обучение

Цикл обучения

Мини-батч: 2048 позиций из последних 500 000 партий

Дообучаем текущую сеть на этих позициях (вход — состояния игры)

Функция потерь сравнивает предсказания сети с поиском и исходом партии

Предсказание	Ошибка	Цель
$\pi_\theta(\cdot \mid s)$	кросс-энтропия	$\pi_{\text{MCTS}}(\cdot \mid s)$
$V_\phi(s)$	MSE	z
	+ регуляризация	

Каждые 1000 циклов — оценка сети

$$\mathcal{L}(\theta, \phi) = (z - V_\phi(s))^2 - \pi_{\text{MCTS}}(\cdot \mid s)^\top \log \pi_\theta(\cdot \mid s) + c\,(\|\theta\|^2 + \|\phi\|^2)$$

MSE критика — кросс-энтропия актора — L2-регуляризация

По мотивам: D. Foster. *AlphaGo Zero Explained In One Diagram*. Medium, 2017; Silver et al. *Mastering the game of Go without human knowledge*. Nature, 2017. doi:10.1038/nature24270.

Как обучается AlphaZero?

1 Игра с собой

Создаём обучающую выборку

Игра против своей копии ⇒ неявный curriculum learning

На каждом ходе сохраняются:



Состояние s_t

π

Вероятности поиска $\pi_{\text{MCTS}}(\cdot \mid s_t)$



Победитель $z \in \{-1, 0, +1\}$ (после партии)

улучшенная поиском стратегия:

$\pi_{\text{MCTS}}(a \mid s_t) \propto N(s_t, a)^{1/\tau}$

2 Дообучение сети

Оптимизируем веса сети обучение

Цикл обучения

Мини-батч: 2048 позиций из последних 500 000 партий

Дообучаем текущую сеть на этих позициях (вход — состояния игры)

Функция потерь сравнивает предсказания сети с поиском и исходом партии

Предсказание	Ошибка	Цель
$\pi_\theta(\cdot \mid s)$	кросс-энтропия	$\pi_{\text{MCTS}}(\cdot \mid s)$
$V_\phi(s)$	MSE	z
	+ регуляризация	

Каждые 1000 циклов — оценка сети

3 Оценка сети

Стала ли новая сеть сильнее?

400 партий: последняя сеть против текущей лучшей

Оба игрока выбирают ходы с помощью MCTS, листья оценивает своя сеть

Последняя сеть становится новой лучшей, если выиграла не менее 55 % партий



Оценка стратегии (игра против предыдущей копии)

$$\mathcal{L}(\theta, \phi) = (z - V_\phi(s))^2 - \pi_{\text{MCTS}}(\cdot \mid s)^\top \log \pi_\theta(\cdot \mid s) + c\left(\|\theta\|^2 + \|\phi\|^2\right)$$

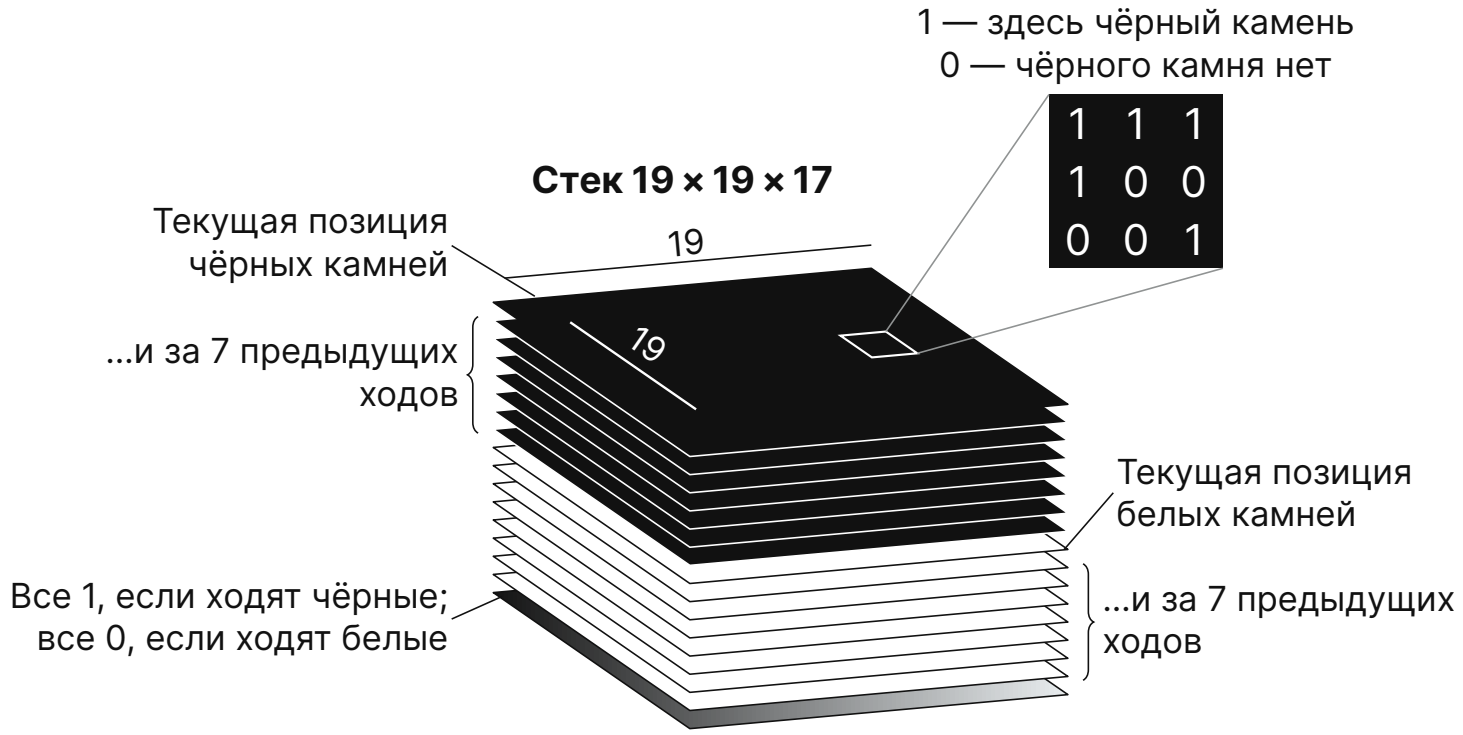
MSE критика — кросс-энтропия актора — L2-регуляризация

По мотивам: D. Foster. *AlphaGo Zero Explained In One Diagram*. Medium, 2017; Silver et al. *Mastering the game of Go without human knowledge*. Nature, 2017. doi:10.1038/nature24270.

Кодировщик ResNet

Кодирование состояния игры

Что такое «состояние игры»?



Этот стек — вход глубокой нейросети

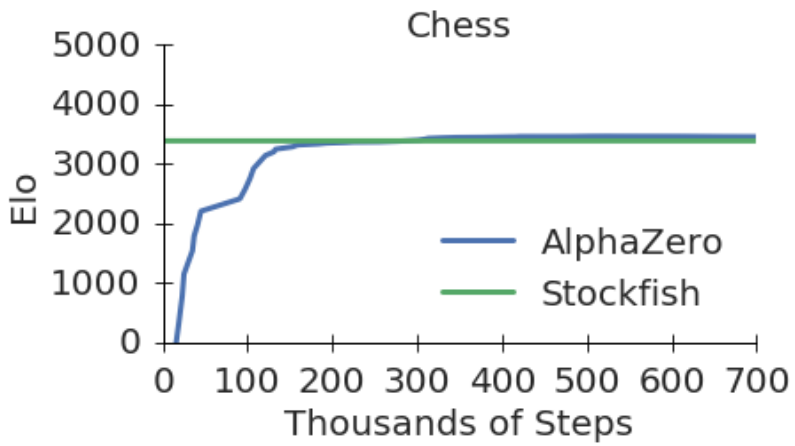
Остаточный блок



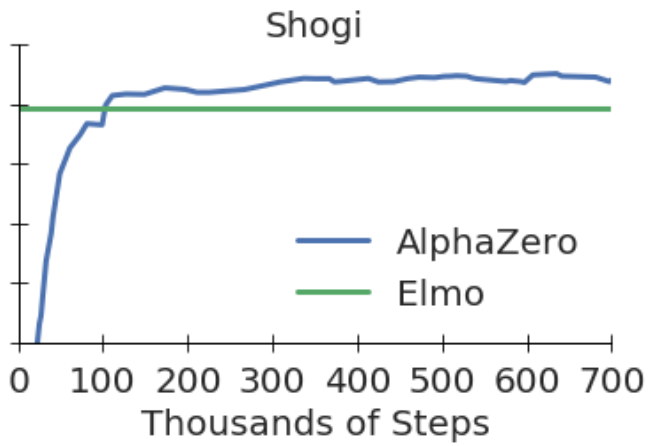
Обычный ResNet без max-pooling

По мотивам: D. Foster. *AlphaGo Zero Explained In One Diagram*. Medium, 2017. Архитектура: Silver et al. *Mastering the game of Go without human knowledge*. Nature, 2017. doi:10.1038/nature24270.

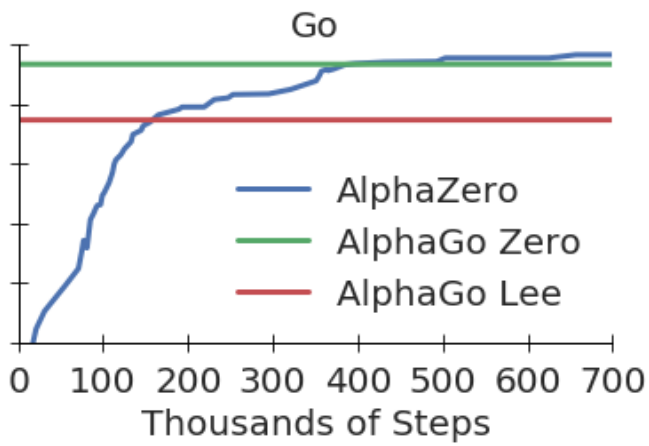
AlphaZero: результаты



Шахматы: Stockfish,
чемпион TCEC 2016



Сёги: Elmo,
чемпион CSA 2017

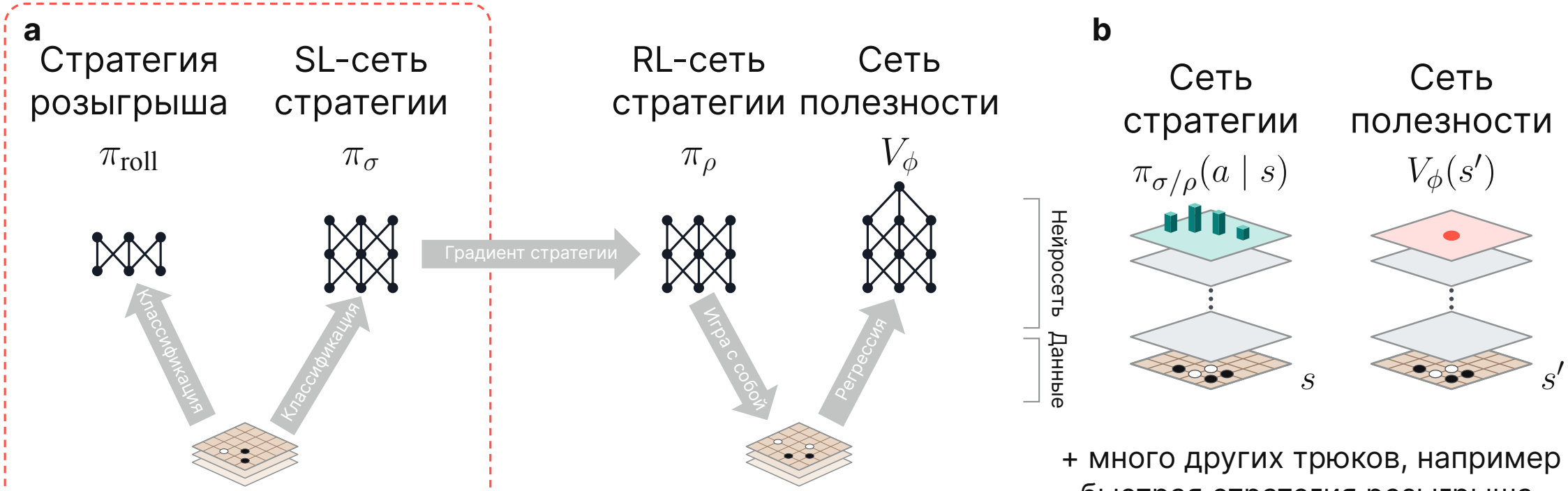


Го: AlphaGo Lee и AlphaGo Zero
(20 блоков, 3 дня обучения)

Обучение AlphaZero: 700 тыс. шагов. Elo вычислен по партиям
между программами при 1 секунде на ход.

Silver et al. *Mastering Chess and Shogi by Self-Play with a General Reinforcement Learning Algorithm*. arXiv, 2017. arXiv:1712.01815.

Отличия от AlphaGo



Silver et al. *Mastering the game of Go with deep neural networks and tree search*. Nature, 2016. doi:10.1038/nature16961.

AlphaZero: итоги

- Обучение с нуля (tabula rasa) игрой с собой
- Без человеческих знаний
- Архитектура кодировщика — ResNet
- Сверхчеловеческий уровень
  в шахматах, сёги и го
- Ограничения?



Обложка *Science*, 7 декабря 2018.

AlphaZero: итоги

- Обучение с нуля (tabula rasa) игрой с собой
- Без человеческих знаний
- Архитектура кодировщика — ResNet
- Сверхчеловеческий уровень
  в шахматах, сёги и го
- Нужен симулятор
- Только детерминированные среды
- Упрощённый RL (нет дисконтирования и т. п.)
- Требует больших вычислений



Обложка *Science*, 7 декабря 2018.

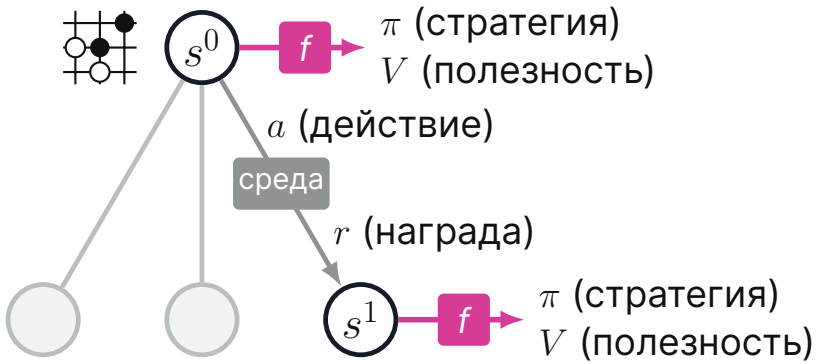
03

MuZero



AlphaZero vs MuZero

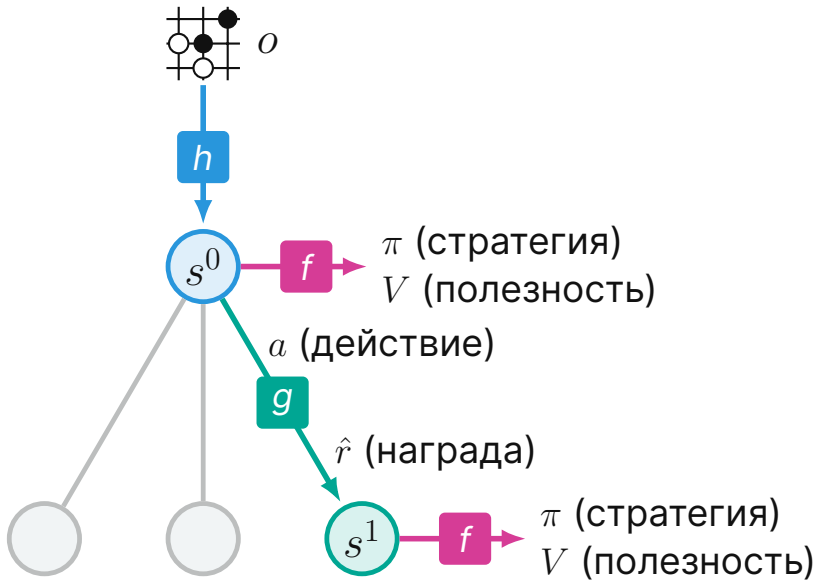
AlphaZero



AlphaZero: 1 сеть

предсказание f: $s \to \pi, V$

MuZero



MuZero: 3 сети

предсказание f: $s \to \pi, V$

динамика g: $s, a \to \hat{r}, s$

представление h: $o \to s$

По мотивам: D. Foster. *How To Build Your Own MuZero AI Using Python (Part 1/3)*. Medium, 2019; Schrittwieser et al. *Mastering Atari, Go, Chess and Shogi by Planning with a Learned Model*. Nature, 2020. arXiv:1911.08265.

MuZero

представление
representation
$h_\theta(o_1, \ldots, o_t) = s^0$

предсказание
prediction
$f_\theta(s^k) = \pi^k, V^k$

динамика
dynamics
$g_\theta(s^{k-1}, a^k) = \hat{r}^k, s^k$

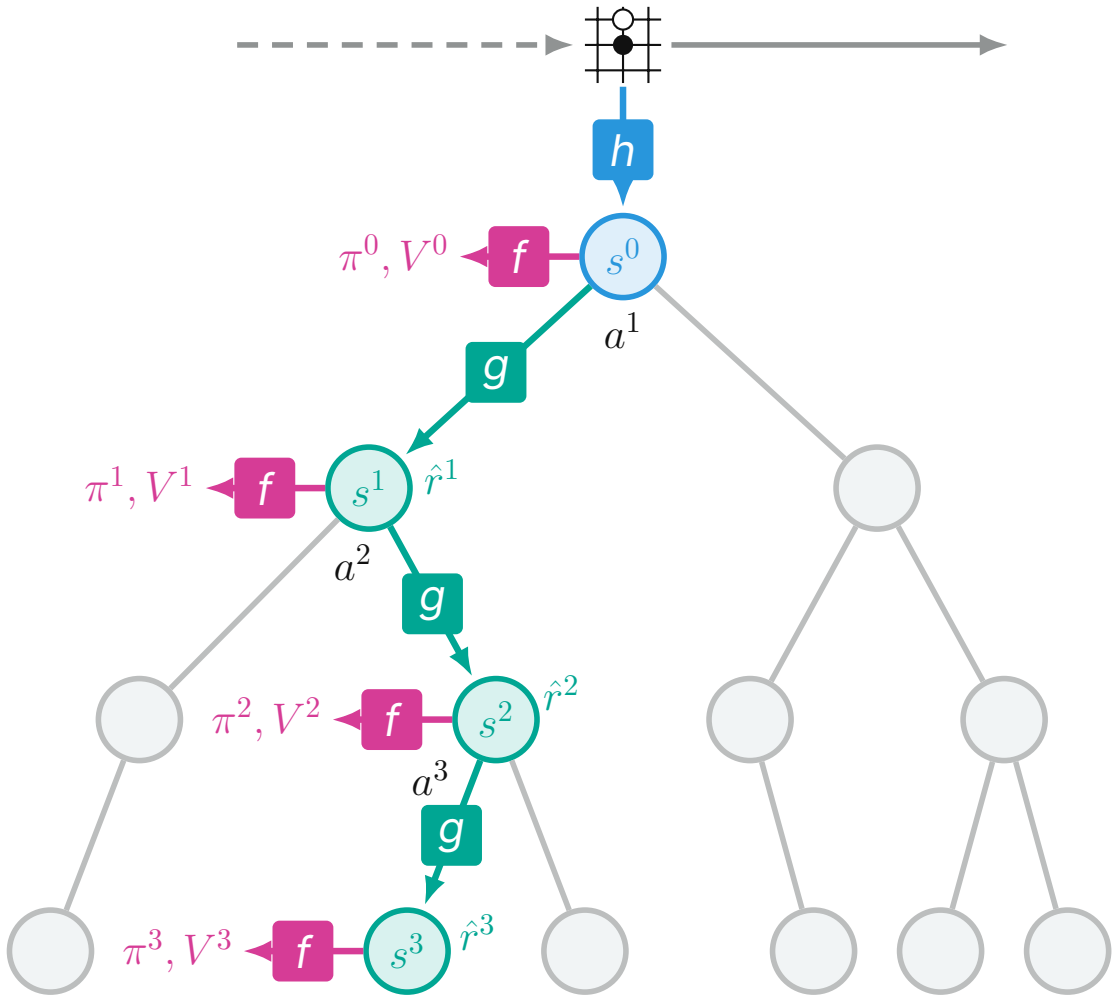


Схема: рис. 1A из Schrittwieser et al. *Mastering Atari, Go, Chess and Shogi by Planning with a Learned Model*. Nature, 2020. arXiv:1911.08265.

pUCT для RL

G — отдача по модели вместо исхода партии

$$W(s,a) \leftarrow W(s,a) + \hat{r}^k + \gamma\,\hat{r}^{k+1} + \cdots + \gamma^{L-k}\,\hat{r}^L + \gamma^{L-k+1}\,V^L$$

для каждого ребра (s^{k-1}, a^k) на пути к листу s^L: награды $\hat{r}$ предсказывает g, полезность листа V^L — f;
$N(s,a)$ и $Q(s,a) = W(s,a)/N(s,a)$ — как в AlphaZero

нормализация

$$\mathrm{pUCT}(s,a) = \frac{Q(s,a) - Q_{\min}}{Q_{\max} - Q_{\min}} + c\,P(s,a)\,\frac{\sqrt{N(s)}}{1 + N(s,a)}$$

$Q_{\min}$, $Q_{\max}$ — наименьшее и наибольшее Q в дереве; дробь лежит в $[0,1]$, как и $P(s,a)$;
нормализация нужна в МППР: в отличие от исхода партии, масштаб наград заранее неизвестен

Schrittwieser et al. *Mastering Atari, Go, Chess and Shogi by Planning with a Learned Model*. Nature, 2020. arXiv:1911.08265.

MuZero

a — как планирует
b — как действует в среде
c — как обучает модель

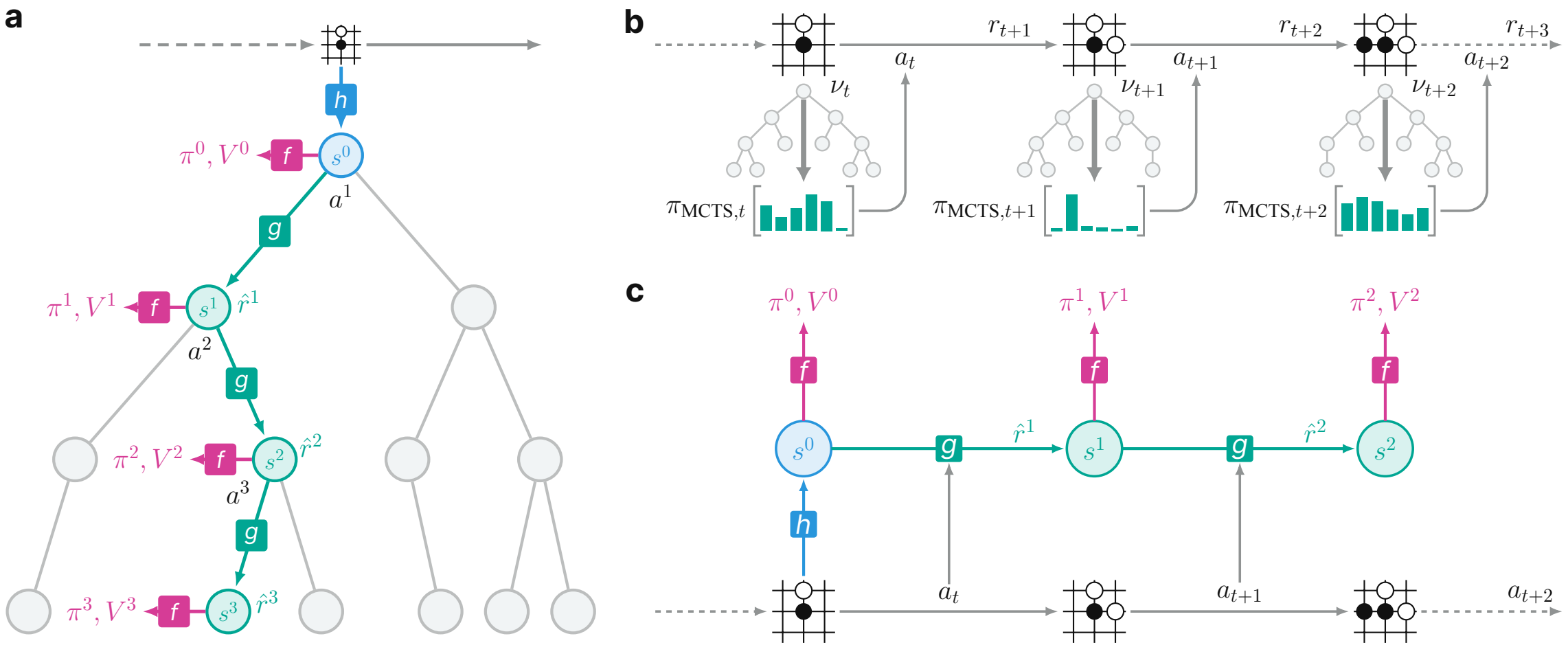


Схема: рис. 1 из Schrittwieser et al. *Mastering Atari, Go, Chess and Shogi by Planning with a Learned Model*. Nature, 2020. arXiv:1911.08265.

Обучение MuZero

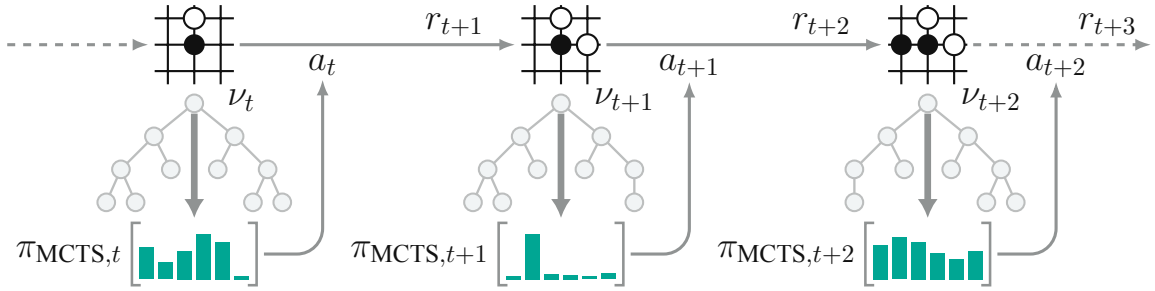
Модель

$$\left.\begin{aligned} s^0 &= h_\theta(o_1, \dots, o_t) \\ \hat{r}^k, s^k &= g_\theta(s^{k-1}, a^k) \\ \pi^k, V^k &= f_\theta(s^k) \end{aligned}\right\} \quad \pi^k, V^k, \hat{r}^k = \mu_\theta(o_1, \dots, o_t,\ a^1, \dots, a^k)$$

h строит s^0, g порождает $\hat{r}^k$ и s^k;
у s^0, s^k нет целей: обучаются неявно

Поиск

$$\nu_t, \pi_{\text{MCTS},t} = \text{MCTS}(s_t^0, \mu_\theta)$$
$$a_t \sim \pi_{\text{MCTS},t}$$



Правило обучения

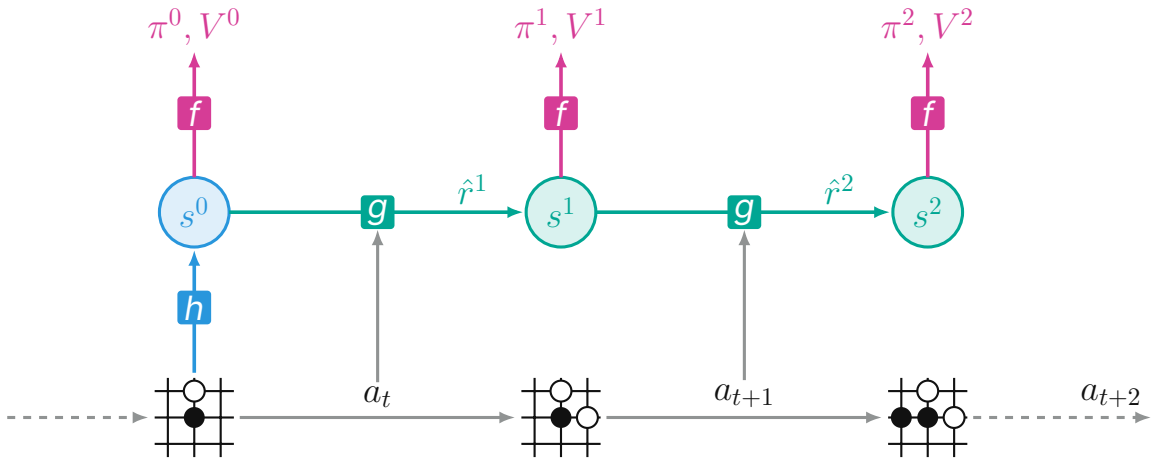
$$\pi_t^k, V_t^k, \hat{r}_t^k = \mu_\theta(o_1, \dots, o_t,\ a_t, \dots, a_{t+k-1})$$

$$z_t = \begin{cases} r_T & \text{для игр} \\ r_{t+1} + \gamma r_{t+2} + \dots + \gamma^{n-1} r_{t+n} + \gamma^n \nu_{t+n} & \text{для общих МППР} \end{cases}$$

n-шаговый бутстреп

$$\ell_t(\theta) = \sum_{k=0}^{K} \ell^r(r_{t+k}, \hat{r}_t^k) + \ell^V(z_{t+k}, V_t^k) + \ell^\pi(\pi_{\text{MCTS},t+k}, \pi_t^k) + c\|\theta\|^2$$

награда — критик — актор — L2-регуляризация



Функции потерь

$$\ell^r(r, \hat{r}) = \begin{cases} 0 & \text{для игр} \\ -\text{bins}(r)^\top \log \hat{\mathbf{r}} & \text{для общих МППР} \end{cases}$$

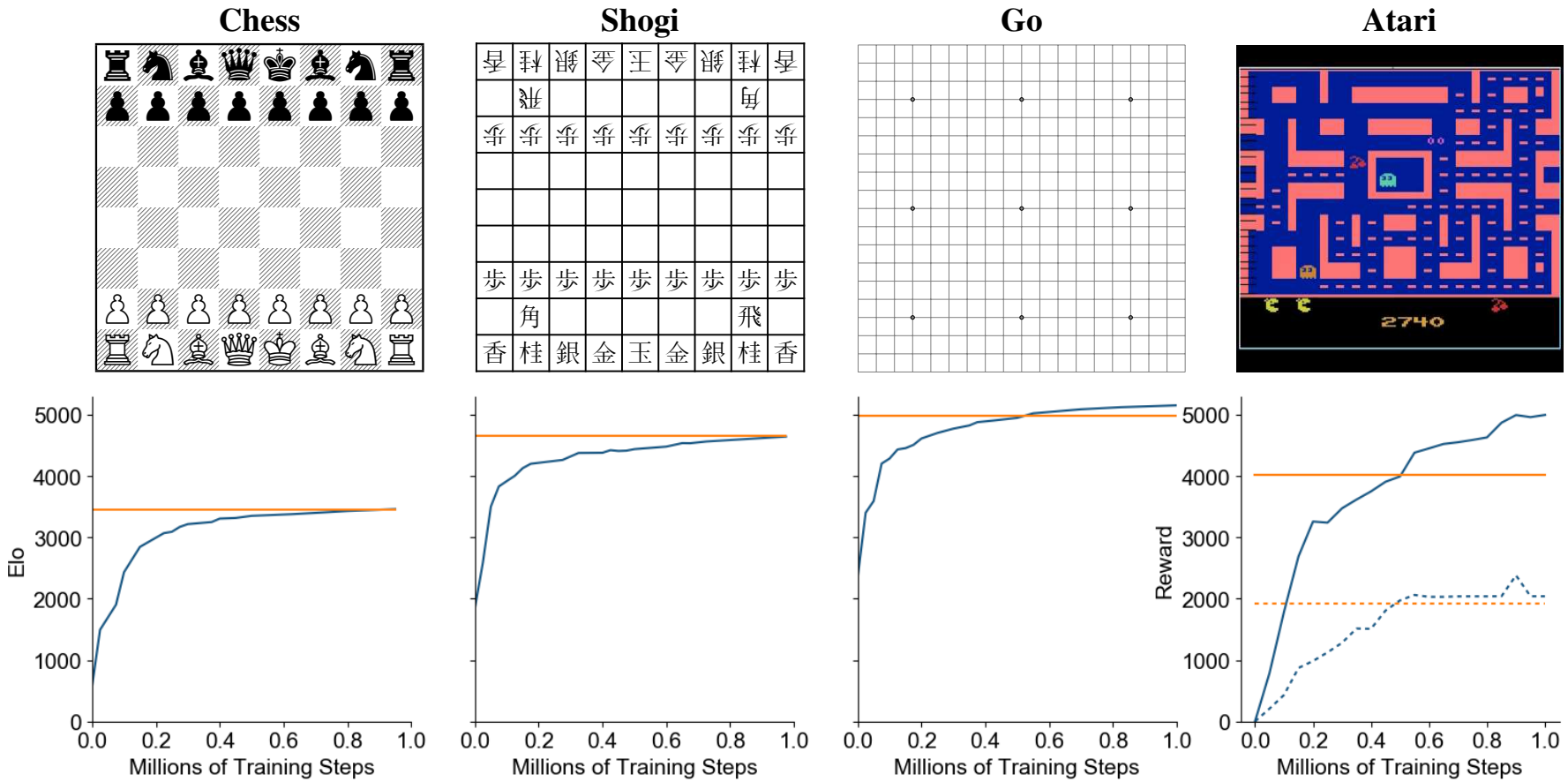
$$\ell^V(z, V) = \begin{cases} (z - V)^2 & \text{для игр} \\ -\text{bins}(z)^\top \log \mathbf{V} & \text{для общих МППР} \end{cases}$$

(кросс-энтропия по бинам)

$$\ell^\pi(\pi, \pi^k) = -\pi^\top \log \pi^k$$

Schrittwieser et al. *Mastering Atari, Go, Chess and Shogi by Planning with a Learned Model*. Nature, 2020. arXiv:1911.08265.

MuZero: результаты



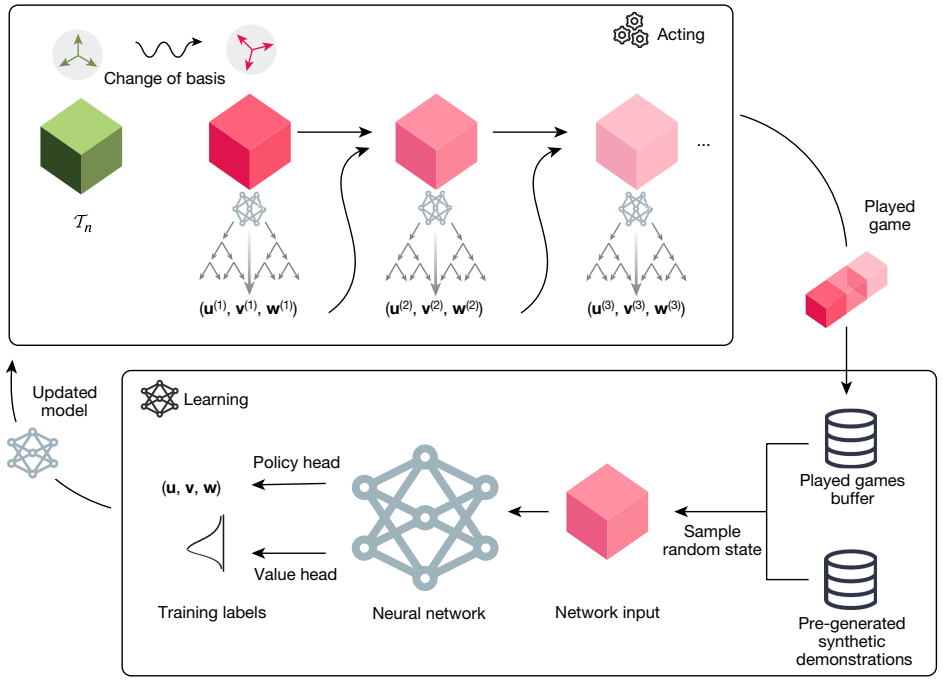
Шахматы, сёги, го: Elo по партиям против AlphaZero; оранжевая линия — сам AlphaZero.
Atari: среднее (сплошная) и медиана (пунктир) по 57 играм в % от человека; оранжевые — R2D2.
Обучение: 1 млн шагов. MuZero не знает правил, но не уступает AlphaZero и бьёт рекорд R2D2.

Schrittwieser et al. *Mastering Atari, Go, Chess and Shogi by Planning with a Learned Model*. Nature, 2020. arXiv:1911.08265.

04

Применения и улучшения

Применения: быстрое умножение матриц



AlphaTensor — 2022

Discovering faster matrix multiplication algorithms with reinforcement learning

Alhussein Fawzi, Matej Balog, Aja Huang, Thomas Hubert, Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, Francisco J. R. Ruiz, Julian Schrittwieser, Grzegorz Swirszcz, David Silver, Demis Hassabis, Pushmeet Kohli

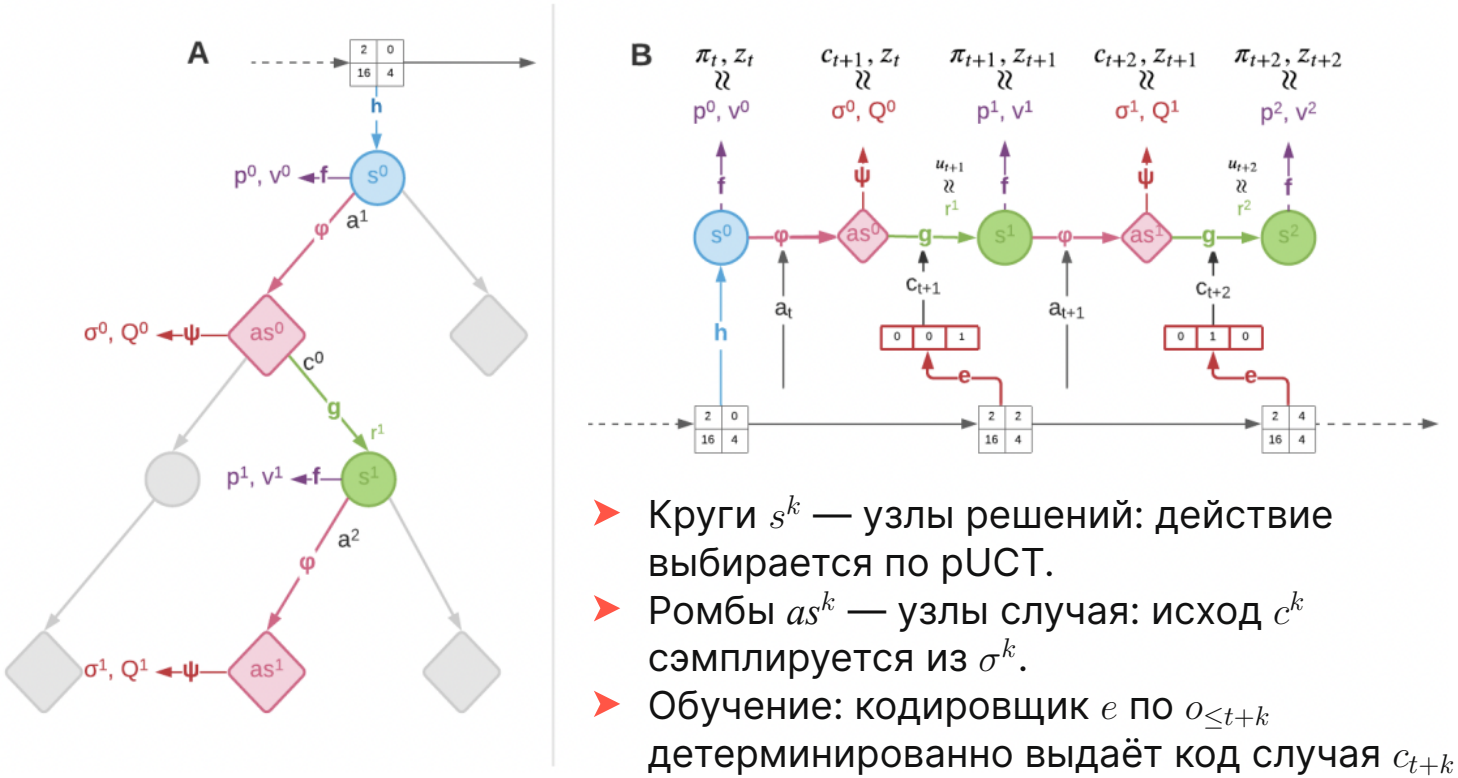
умножение матриц $n \times m$ на $m \times p$

Размер (n, m, p)	Лучший известный метод	Лучший известный ранг	Ранг AlphaTensor	
			$\mathbb{Z}_2$	$\mathbb{R}$
(2, 2, 2)	Strassen, 1969	7	7	7
(3, 3, 3)	Laderman, 1976	23	23	23
(4, 4, 4)	Strassen, 1969 (2, 2, 2) $\otimes$ (2, 2, 2)	49	47	49
(5, 5, 5)	(3, 5, 5) + (2, 5, 5)	98	96	98
(2, 2, 3)	(2, 2, 2) + (2, 2, 1)	11	11	11
(2, 2, 4)	(2, 2, 2) + (2, 2, 2)	14	14	14
(2, 2, 5)	(2, 2, 2) + (2, 2, 3)	18	18	18
(2, 3, 3)	Hopcroft & Kerr, 1971	15	15	15
(2, 3, 4)	Hopcroft & Kerr, 1971	20	20	20
(2, 3, 5)	Hopcroft & Kerr, 1971	25	25	25
(2, 4, 4)	Hopcroft & Kerr, 1971	26	26	26
(2, 4, 5)	Hopcroft & Kerr, 1971	33	33	33
(2, 5, 5)	Hopcroft & Kerr, 1971	40	40	40
(3, 3, 4)	Smirnov, 2013	29	29	29
(3, 3, 5)	Smirnov, 2013	36	36	36
(3, 4, 4)	Smirnov, 2013	38	38	38
(3, 4, 5)	Smirnov, 2013	48	47	47
(3, 5, 5)	Sedoglavic & Smirnov, 2021	58	58	58
(4, 4, 5)	(4, 4, 2) + (4, 4, 3)	64	63	63
(4, 5, 5)	(2, 5, 5) $\otimes$ (2, 1, 1)	80	76	76

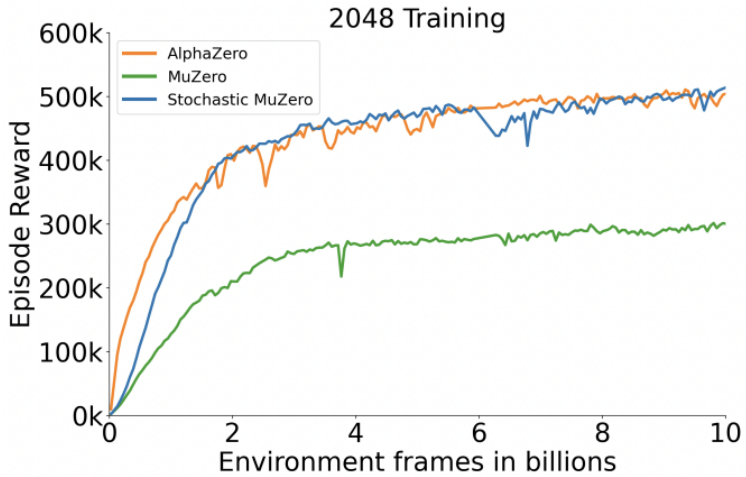
Ранг — число скалярных умножений; красным — улучшения.
$\mathbb{Z}_2$ — арифметика по модулю 2, $\mathbb{R}$ — стандартная.

Fawzi et al. *Discovering faster matrix multiplication algorithms with reinforcement learning*. Nature, 2022. doi:10.1038/s41586-022-05172-4.

Применения: стохастические среды



- Круги s^k — узлы решений: действие выбирается по pUCT.
- Ромбы as^k — узлы случая: исход c^k сэмплируется из σ^k.
- Обучение: кодировщик e по $o_{\leq t+k}$ детерминированно выдаёт код случая c_{t+k} (one-hot), а σ^k учится его предсказывать.



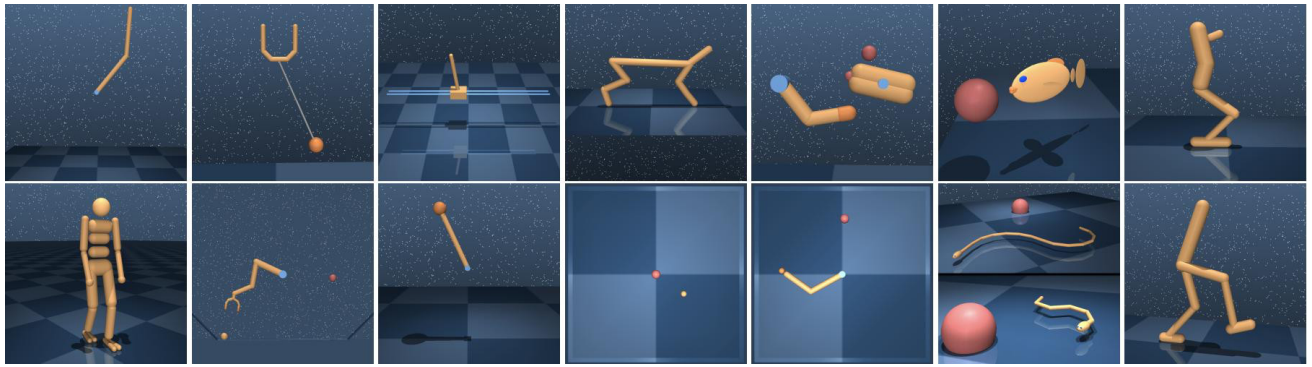


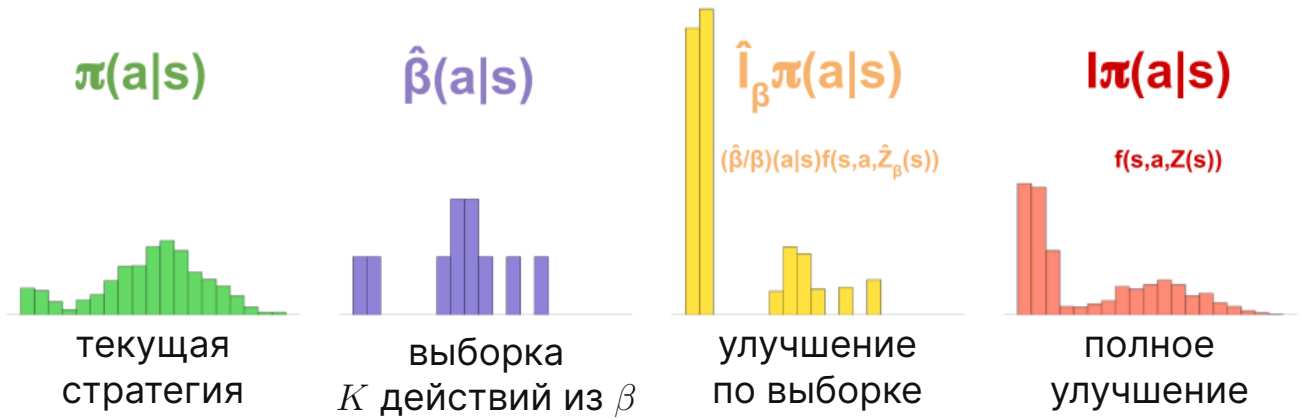
Planning in Stochastic Environments with a Learned Model

Ioannis Antonoglou, Julian Schrittwieser, Sherjil Ozair, Thomas K Hubert, David Silver

Antonoglou et al. *Planning in Stochastic Environments with a Learned Model*. ICLR, 2022. openreview.net/forum?id=X6D9bAHhBQ1.

Применения: непрерывные действия (MuJoCo)





При росте K улучшение по выборке $\hat{\mathcal{I}}_{\beta}\pi$ сходится к полному $\mathcal{I}\pi$

Learning and Planning in Complex Action Spaces

DeepMind Control Suite, обучение по пикселям:
отдача после 5 млн шагов среды

Задача	Dreamer	SMuZero
acrobot.swingup	365.26	**417.52**
cartpole.balance	**979.56**	**984.86**
cartpole.balance_sparse	941.84	**998.14**
cartpole.swingup	833.66	**868.87**
cartpole.swingup_sparse	812.22	**846.91**
cheetah.run	894.56	**914.39**
ball_in_cup.catch	962.48	**977.38**
finger.spin	498.88	**986.38**
finger.turn_easy	825.86	**972.53**
finger.turn_hard	891.38	**963.07**
hopper.hop	368.97	**528.24**
hopper.stand	**923.72**	**926.50**
pendulum.swingup	**833.00**	**837.76**
quadruped.run	888.39	**923.54**
quadruped.walk	**931.61**	**933.77**
reacher.easy	935.08	**982.26**
reacher.hard	817.05	**971.53**
walker.run	824.67	**931.06**
walker.stand	**977.99**	**987.79**
walker.walk	961.67	**975.46**

SMuZero не хуже Dreamer во всех 20 задачах

Hubert et al. *Learning and Planning in Complex Action Spaces*. ICML, 2021. arXiv:2104.06303.

Применения: офлайн RL

Алгоритм 2. Reanalyse

Вход: $\mathcal{D}$ — сохранённые траектории: o_t, a_t, r_{t+1}

for шаг $\leftarrow 1, 2, \ldots$ **do**

$t \leftarrow$ случайный шаг траектории из $\mathcal{D}$

$s_t^0 \leftarrow h_\theta(o_1, \ldots, o_t)$

for $k \leftarrow 0, \ldots, K$ **do**

$\pi_{\mathrm{MCTS},t+k},\ \nu_{t+k} \leftarrow \mathrm{MCTS}\big(h_\theta(o_1, \ldots, o_{t+k}),\ \theta\big)$

$\pi_t^k, V_t^k \leftarrow f_\theta(s_t^k)$

$\hat{r}_t^{k+1}, s_t^{k+1} \leftarrow g_\theta(s_t^k, a_{t+k})$

свежие цели: повторный поиск с текущими θ

$\ell \leftarrow \sum_{k=1}^{K} \ell^r(r_{t+k}, \hat{r}_t^k)$

$\quad + \sum_{k=0}^{K} \big[\ell^V(z_{t+k}, V_t^k) + \ell^\pi(\pi_{\mathrm{MCTS},t+k}, \pi_t^k)\big]$

$\theta \leftarrow \theta - \alpha \nabla_\theta \ell$

Цели обновляются вместе с моделью, без новых действий. MuZero Unplugged — 100 % Reanalyse, только по $\mathcal{D}$.

Игра	QR-DQN	REM	CQL(H)	MZ
asterix (1 %)	166.3	386.5	592.4	**27220.5**
breakout	7.9	11.0	61.1	**251.9**
pong	−13.8	−6.9	**19.3**	−16.2
qbert	383.6	343.4	**14012.0**	6953.2
seaquest	672.9	499.8	779.4	**4964.0**
asterix (10 %)	189.2	75.1	156.3	**40554.0**
breakout	151.2	86.7	269.3	**485.8**
pong	15.1	8.9	**18.5**	15.6
qbert	7091.3	8624.3	13855.6	**16817.9**
seaquest	2984.8	3936.6	3674.1	**8556.3**

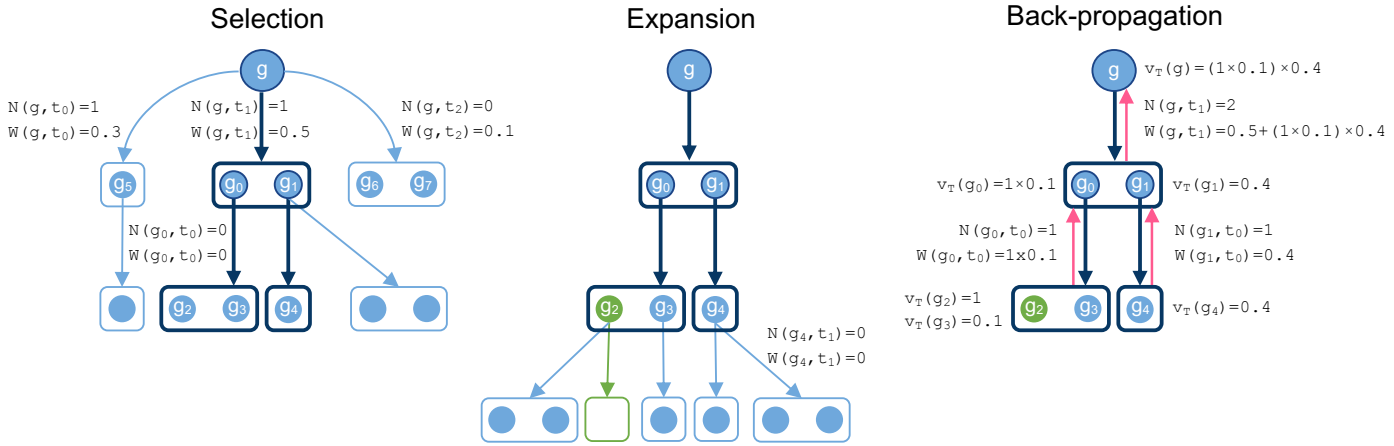
Atari, мало данных: очки в играх после обучения лишь на 1 % (сверху, 2 млн кадров) или 10 % (снизу, 20 млн кадров) данных Atari. MZ — MuZero Unplugged, жирным — лучший в строке. С ростом объёма данных MuZero Unplugged стабильно улучшается.

Online and Offline Reinforcement Learning by Planning with a Learned Model

Julian Schrittwieser, Thomas Hubert, Amol Mandhane, Mohammadamin Barekatain, Ioannis Antonoglou, David Silver

Schrittwieser et al. *Online and Offline Reinforcement Learning by Planning with a Learned Model*. NeurIPS, 2021. arXiv:2104.06294.

Применения: доказательство теорем (HTPS)



Онлайн-обучение на задачах	С учителем —	GPT-f miniF2F-curriculum	Evariste miniF2F-valid
miniF2F-valid	38.5	47.3	58.6†
miniF2F-test	35.3	36.6	41.0
miniF2F-curriculum	20.8	30.6	32.1
Время обучения, дней A100	50	2000	1360

Тактика сводит цель к набору подцелей, и доказать нужно их все. Поэтому выбор по pUCT (как в AlphaZero) строит не путь, а гипердерево (тёмно-синее), и раскрываются сразу все его нераскрытые листья.

Полезность v_T листа: 1 (цель доказана), 0 (тупик) или оценка критика; у родителя — произведение v_T подцелей. Счётчики — как в MCTS (цель g — состояние, тактика t — действие): $N(g,t) \leftarrow N(g,t)+1$, $W(g,t) \leftarrow W(g,t)+v_T(g)$.

Lean, pass@64: доля теорем (%), доказанных хотя бы в одном из 64 запусков поиска.

† Эти теоремы участвуют в онлайн-обучении; дана доля доказанных хотя бы раз за обучение.

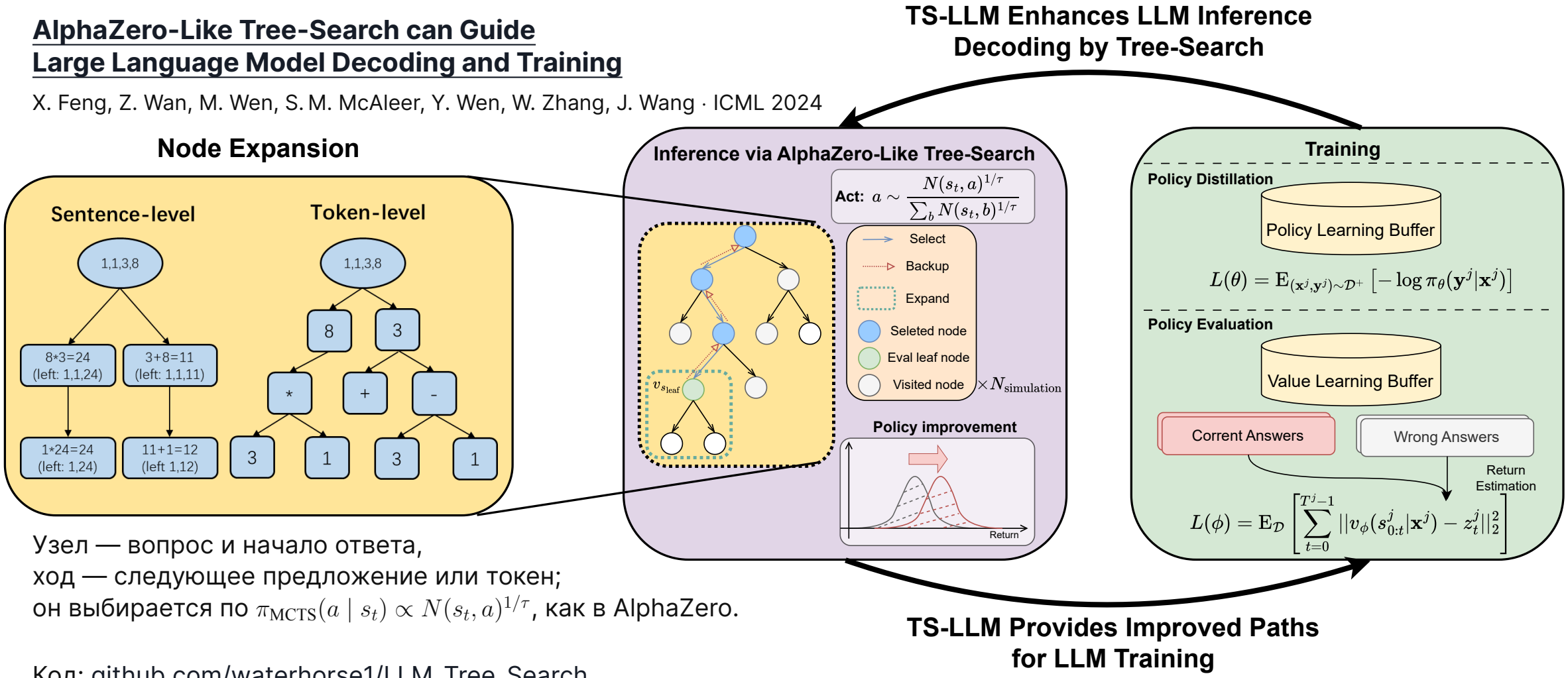
Evariste — метод статьи: поиск HTPS, а сети стратегии и критика учатся онлайн на его данных. На miniF2F-test, не участвующем в обучении: 41.0 против 36.6 у GPT-f.

Lample et al. *HyperTree Proof Search for Neural Theorem Proving*. NeurIPS, 2022. arXiv:2205.11491.

Применения: AlphaZero для LLM

AlphaZero-Like Tree-Search can Guide Large Language Model Decoding and Training

X. Feng, Z. Wan, M. Wen, S. M. McAleer, Y. Wen, W. Zhang, J. Wang · ICML 2024



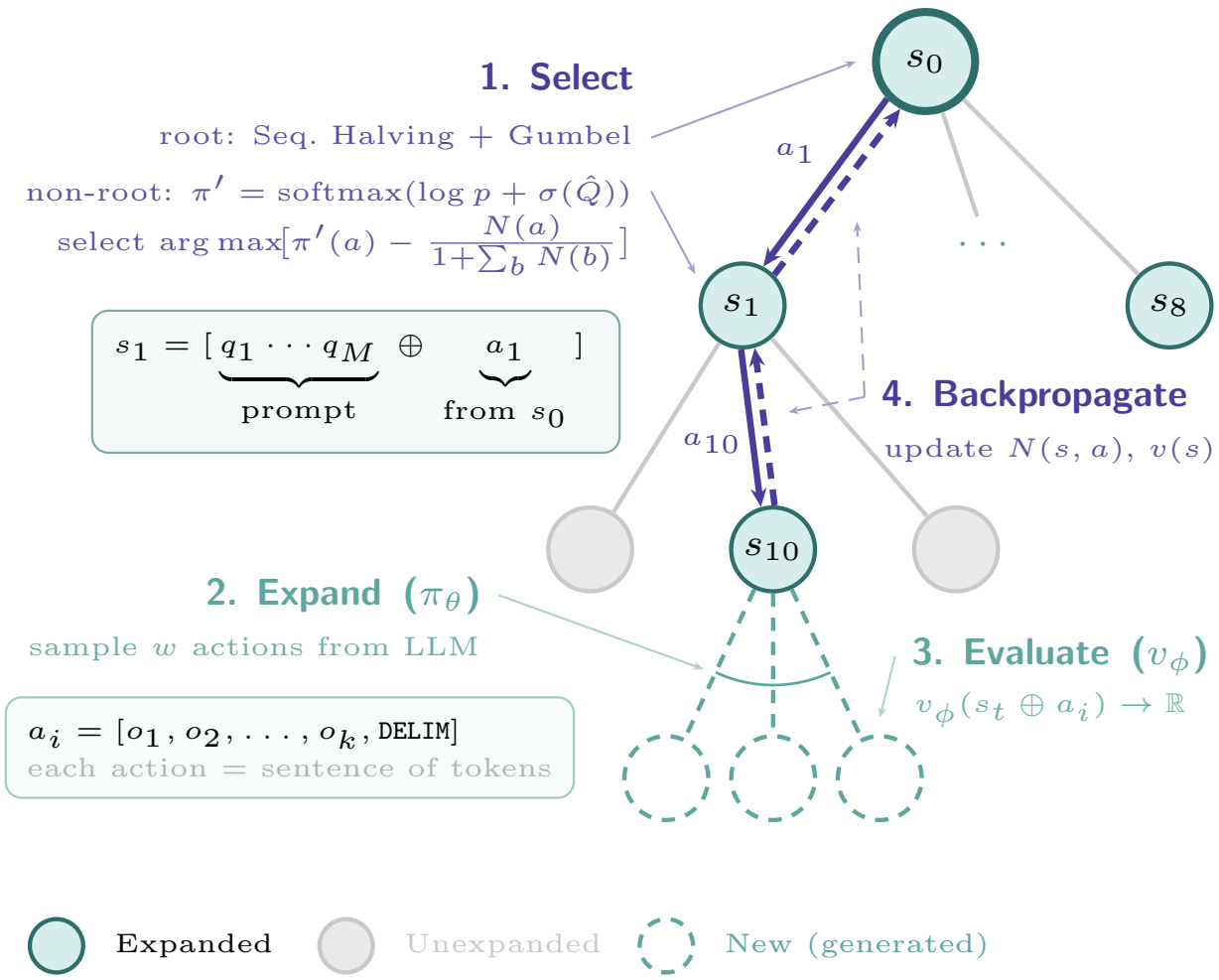
Узел — вопрос и начало ответа,
ход — следующее предложение или токен;
он выбирается по $\pi_{\text{MCTS}}(a \mid s_t) \propto N(s_t, a)^{1/\tau}$, как в AlphaZero.

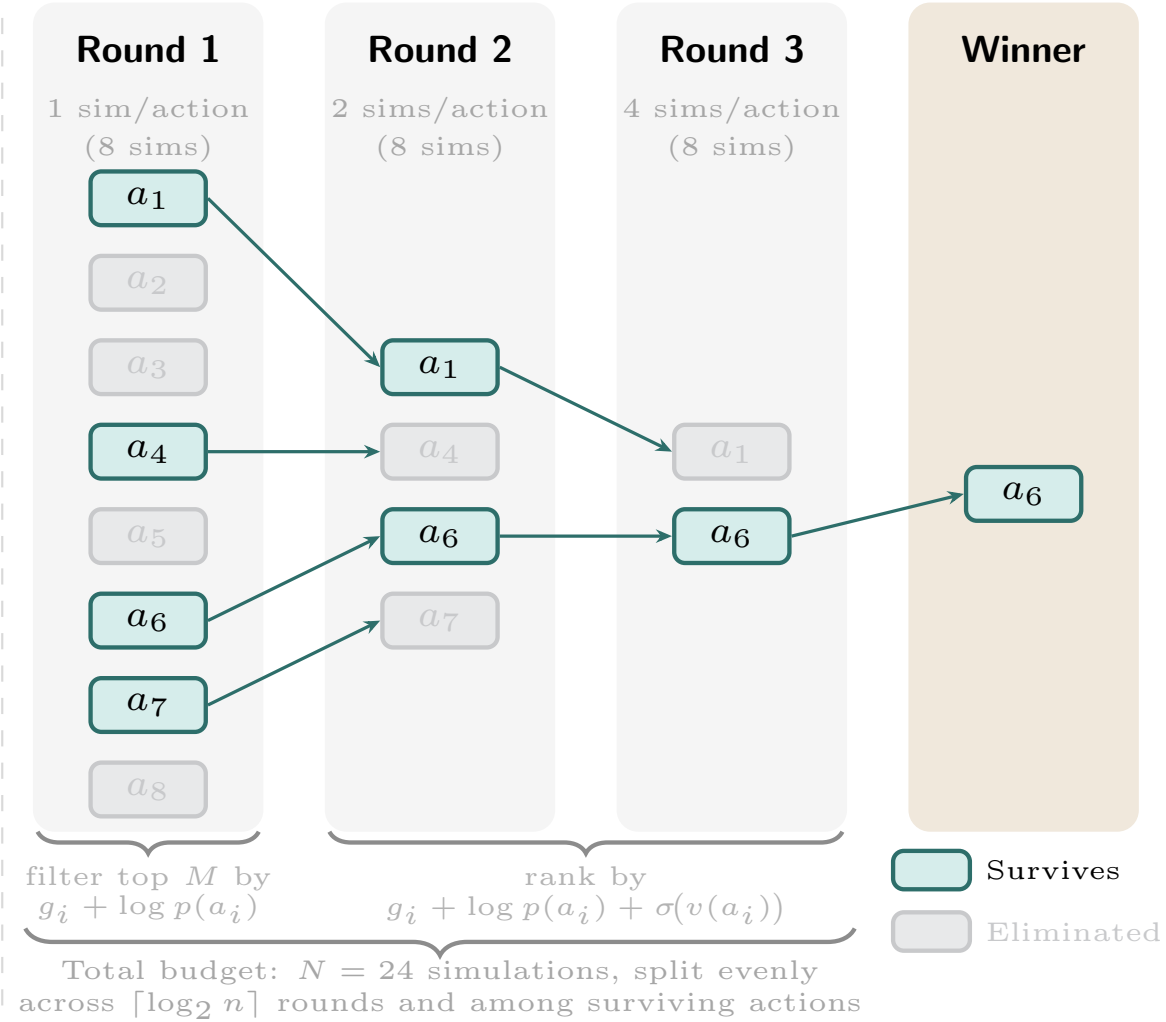
Код: github.com/waterhorse1/LLM_Tree_Search

Feng et al. *AlphaZero-Like Tree-Search can Guide Large Language Model Decoding and Training*. ICML, 2024. arXiv:2309.17179.

Revisiting AlphaZero для LLM

Revisiting Tree Search for LLMs: Gumbel and Sequential Halving for Budget-Scalable Reasoning Ugadiarov, Kuratov, Panov, Skrynnik, 2026

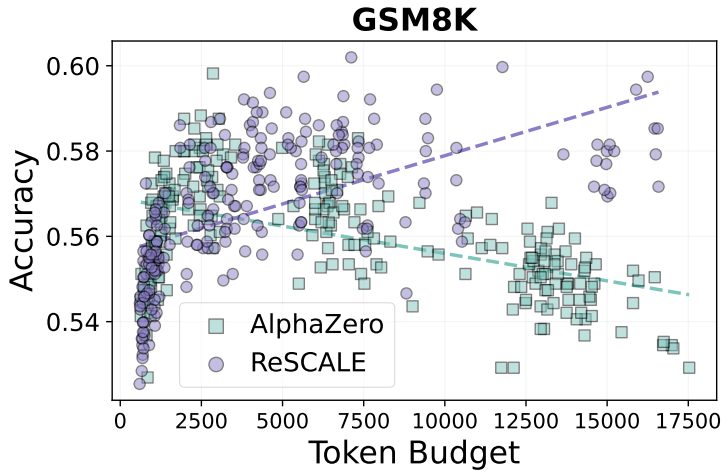




Ugadiarov et al. *Revisiting Tree Search for LLMs: Gumbel and Sequential Halving for Budget-Scalable Reasoning*. arXiv, 2026. arXiv:2603.21162.

Revisiting AlphaZero для LLM

Вывод: мы проанализировали деревья MCTS для примеров, в которых AlphaZero MCTS давал неверные ответы, и обнаружили, что поиск чрезмерно эксплуатирует небольшое подмножество действий. Из-за этого возникает сильный дисбаланс числа посещений между исследованными и недоисследованными узлами. С ростом бюджета проблема усугубляется: выбор в корне отдаёт предпочтение самому посещаемому узлу и игнорирует потенциально лучшие альтернативы. ReSCALE смягчает её с помощью Sequential Halving, сохраняя разнообразие исследования на каждом шаге.

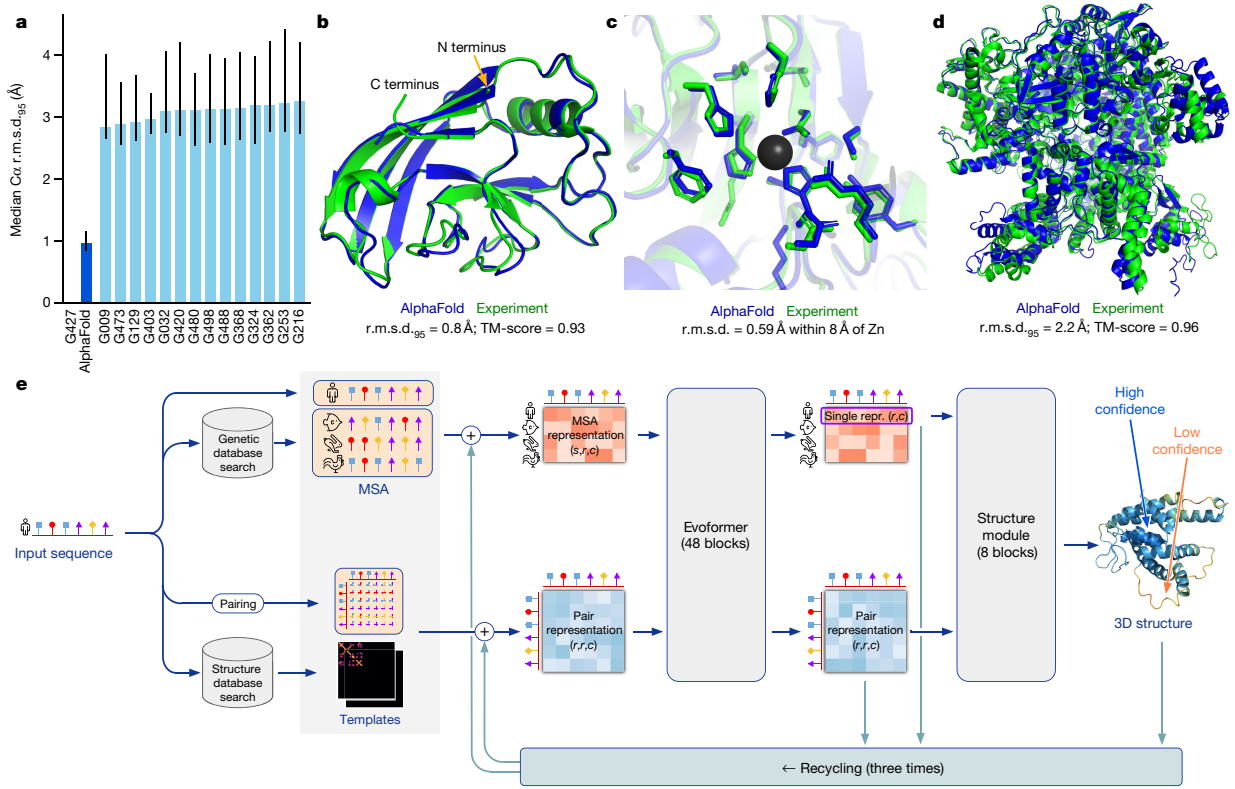


		GSM8K			Game24		
Поиск	Бюджет	Токены	Точн., %	Макс. точн., %	Токены	Точн., %	Макс. точн., %
AlphaZero	Малый	0.5K–2K	56.0 ± 1.2	58.8	0.2K–2K	74.4 ± 12.0	86.7
ReSCALE			55.1 ± 1.1	58.6		71.6 ± 10.5	81.2
AlphaZero	Средний	6K–8K	56.7 ± 0.9	58.3	2K–4K	84.3 ± 1.0	86.2
ReSCALE			57.9 ± 1.0	60.2		83.4 ± 1.4	85.9
AlphaZero	Большой	16K–18K	53.6 ± 0.7	55.0	4K–6K	82.9 ± 1.1	84.8
ReSCALE			58.4 ± 0.9	59.7		85.3 ± 0.6	85.9
Best-of-N	N = 32	3.5K	53.4 ± 0.4	—	2.5K	54.1 ± 1.0	—

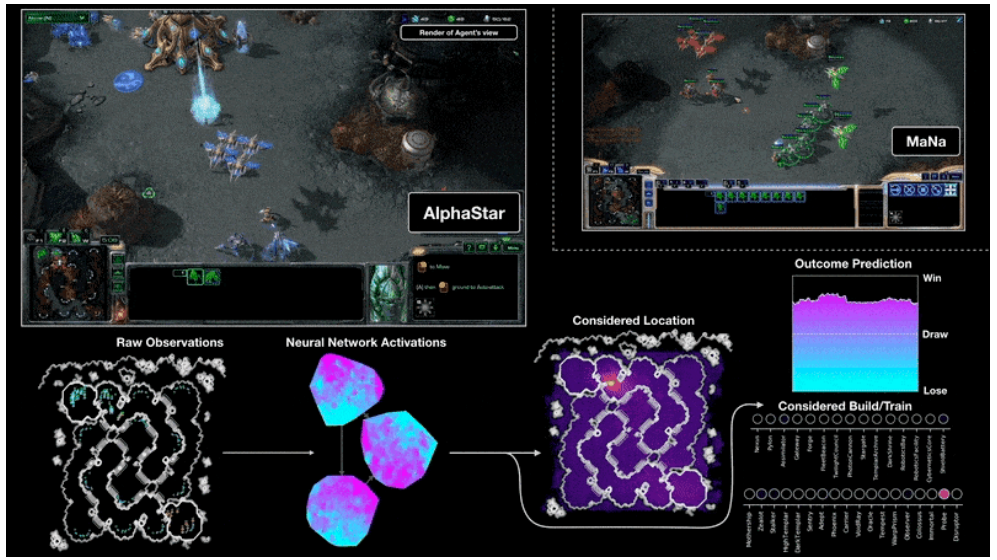
Ugadiarov et al. *Revisiting Tree Search for LLMs: Gumbel and Sequential Halving for Budget-Scalable Reasoning*. arXiv, 2026. arXiv:2603.21162.

Ложные AlphaZero

AlphaFold и AlphaStar — не AlphaZero :)



AlphaFold 2, 2021. Обучение с учителем на структурах из PDB и самодистилляция; сеть напрямую выдаёт 3D-координаты атомов. Ни RL, ни поиска по дереву.



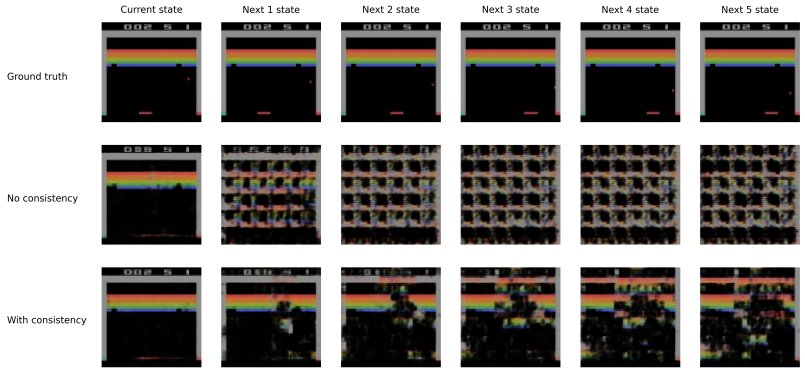
AlphaStar, 2019. Обучение с учителем на играх людей, затем multi-agent RL в лиге агентов. RL есть, но поиска по дереву нет: действие выдаёт сеть.

Jumper et al. *Highly accurate protein structure prediction with AlphaFold*. Nature, 2021. doi:10.1038/s41586-021-03819-2.
Vinyals et al. *Grandmaster level in StarCraft II using multi-agent reinforcement learning*. Nature, 2019. doi:10.1038/s41586-019-1724-z.

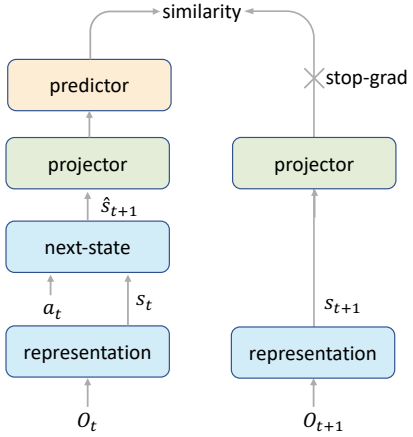
Открытые реализации: EfficientZero

EfficientZero	норм. очки, 26 игр среднее	медиана
полная	**1.943**	**1.090**
без согласованности	0.881	0.340
без value prefix	1.482	0.552
без off-policy коррекции	1.475	0.836

Сильнее всего падает без согласованности



Декодер восстанавливает кадры из скрытых состояний: текущего и предсказанных на 1–5 шагов. Без согласованности — лишь из текущего.



Выход динамики g_θ (next-state) подтягивается к представлению h_θ следующего реального наблюдения o_{t+1}, и так на 5 шагов вперёд.

Как в SimSiam: целевая ветвь — без градиента (stop-grad).

Game	Random	Human	SimPLe	OTRainbow	CURL	DrQ	SPR	MuZero	Ours
Alien	227.8	7127.7	616.9	**824.7**	558.2	771.2	801.5	530.0	808.5
Amidar	5.8	1719.5	88.0	82.8	142.1	102.8	**176.3**	38.8	148.6
Assault	222.4	742.0	527.2	351.9	600.6	452.4	571.0	500.1	**1263.1**
Asterix	210.0	8503.3	1128.3	628.5	734.5	603.5	977.8	1734.0	**25557.8**
Bank Heist	14.2	753.1	34.2	182.1	131.6	168.9	**380.9**	192.5	351.0
BattleZone	2360.0	37187.5	5184.4	4060.6	14870.0	12954.0	**16651.0**	7687.5	13871.2
Boxing	0.1	12.1	9.1	2.5	1.2	6.0	35.8	15.1	**52.7**
Breakout	1.7	30.5	16.4	9.8	4.9	16.1	17.1	48.0	**414.1**
ChopperCmd	811.0	7387.8	1246.9	1033.3	1058.5	780.3	974.8	**1350.0**	1117.3
Crazy Climber	10780.5	35829.4	62583.6	21327.8	12146.5	20516.5	42923.6	56937.0	**83940.2**
Demon Attack	152.1	1971.0	208.1	711.8	817.6	1113.4	545.2	3527.0	**13003.9**
Freeway	0.0	29.6	20.3	25.0	**26.7**	9.8	24.4	21.8	21.8
Frostbite	65.2	4334.7	254.7	231.6	1181.3	331.1	**1821.5**	255.0	296.3
Gopher	257.6	2412.5	771.0	778.0	669.3	636.3	715.2	1256.0	**3260.3**
Hero	1027.0	30826.4	2656.6	6458.8	6279.3	3736.3	7019.2	3095.0	**9315.9**
Jamesbond	29.0	302.8	125.3	112.3	471.0	236.0	365.4	87.5	**517.0**
Kangaroo	52.0	3035.0	323.1	605.4	872.5	940.6	**3276.4**	62.5	724.1
Krull	1598.0	2665.5	4539.9	3277.9	4229.6	4018.1	3688.9	4890.8	**5663.3**
Kung Fu Master	258.5	22736.3	17257.2	5722.2	14307.8	9111.0	13192.7	18813.0	**30944.8**
Ms Pacman	307.3	6951.6	**1480.0**	941.9	1465.5	960.5	1313.2	1265.6	1281.2
Pong	-20.7	14.6	12.8	1.3	-16.5	-8.5	-5.9	-6.7	**20.1**
Private Eye	24.9	69571.3	58.3	100.0	**218.4**	-13.6	124.0	56.3	96.7
Qbert	163.9	13455.0	1288.8	509.3	1042.4	854.4	669.1	3952.0	**13781.9**
Road Runner	11.5	7845.0	5640.6	2696.7	5661.0	8895.1	14220.5	2500.0	**17751.3**
Seaquest	68.4	42054.7	683.3	286.9	384.5	301.2	583.1	208.0	**1100.2**
Up N Down	533.4	11693.2	3350.3	2847.6	2955.2	3180.8	**28138.5**	2896.9	17264.2
Normed Mean	0.000	1.000	0.443	0.264	0.381	0.357	0.704	0.562	**1.943**
Normed Median	0.000	1.000	0.144	0.204	0.175	0.268	0.415	0.227	**1.090**

Atari 100k: 26 игр, по 100 тыс. шагов среды (≈ 2 ч игры); Ours — EfficientZero, жирным — лучший алгоритм; Normed — (агент − Random) / (Human − Random).

Впервые агент, обученный на 2 ч игры, обходит человека по среднему и медиане.

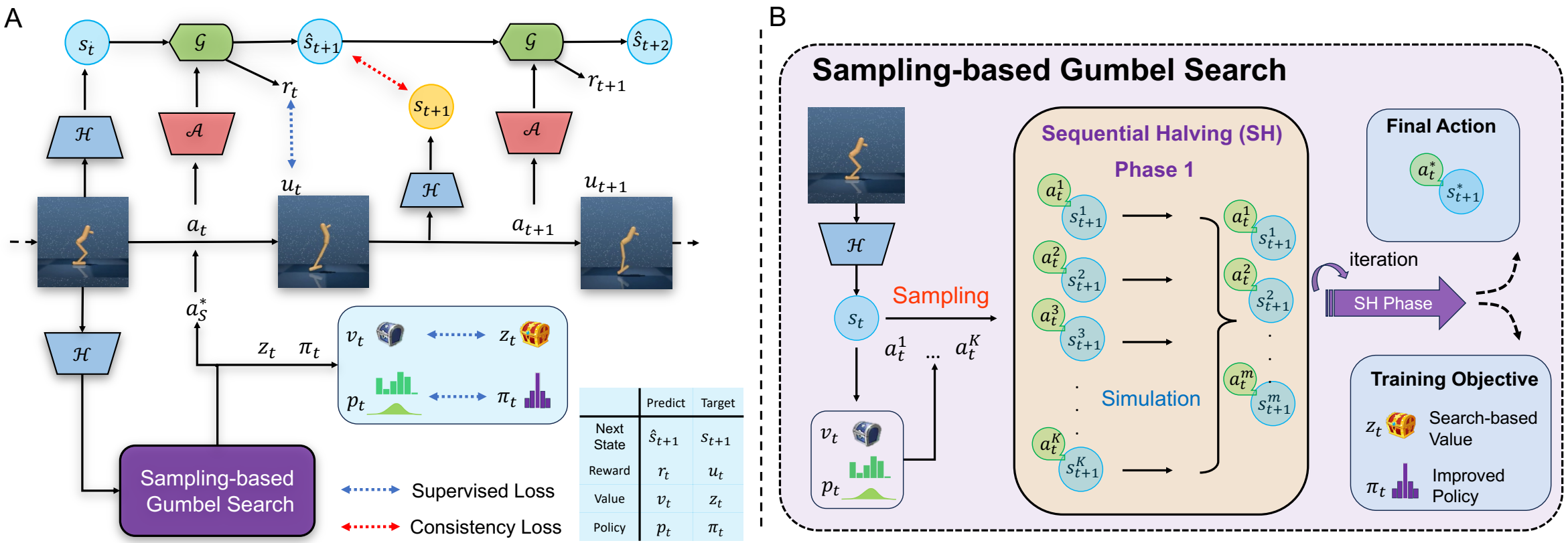
Mastering Atari Games with Limited Data

Weirui Ye, Shaohuai Liu, Thanard Kurutach, Pieter Abbeel, Yang Gao

Код: github.com/YeWR/EfficientZero

Ye et al. *Mastering Atari Games with Limited Data*. NeurIPS, 2021. arXiv:2111.00210.

Открытые реализации: EfficientZero V2



EfficientZero V2: Mastering Discrete and Continuous Control with Limited Data

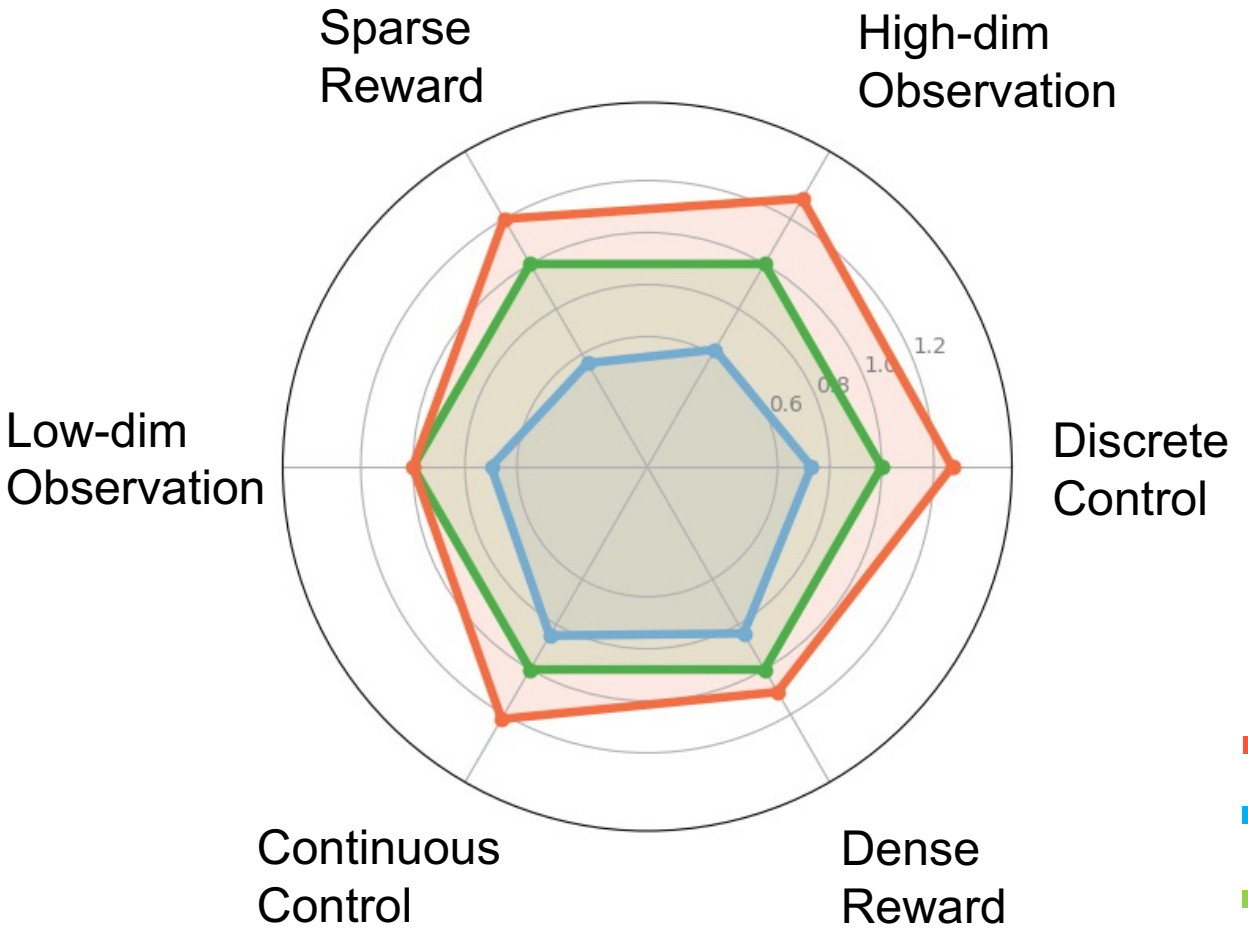
S. Wang, S. Liu, W. Ye, J. You, Y. Gao · ICML 2024

Код: github.com/Shengjiewang-Jason/EfficientZeroV2

Отличия от EfficientZero: поиск Gumbel (как в ReSCALE) — Sequential Halving по выборке из K действий, в том числе непрерывных; для старых переходов цель z_t даёт сам поиск по текущей модели.

Wang et al. *EfficientZero V2: Mastering Discrete and Continuous Control with Limited Data*. ICML, 2024. arXiv:2403.00564.

Открытые реализации: EfficientZero V2



66 задач: Atari 100k (26 игр), DMControl Proprio и Vision (по 20); бюджет — от 50 до 200 тыс. взаимодействий со средой.

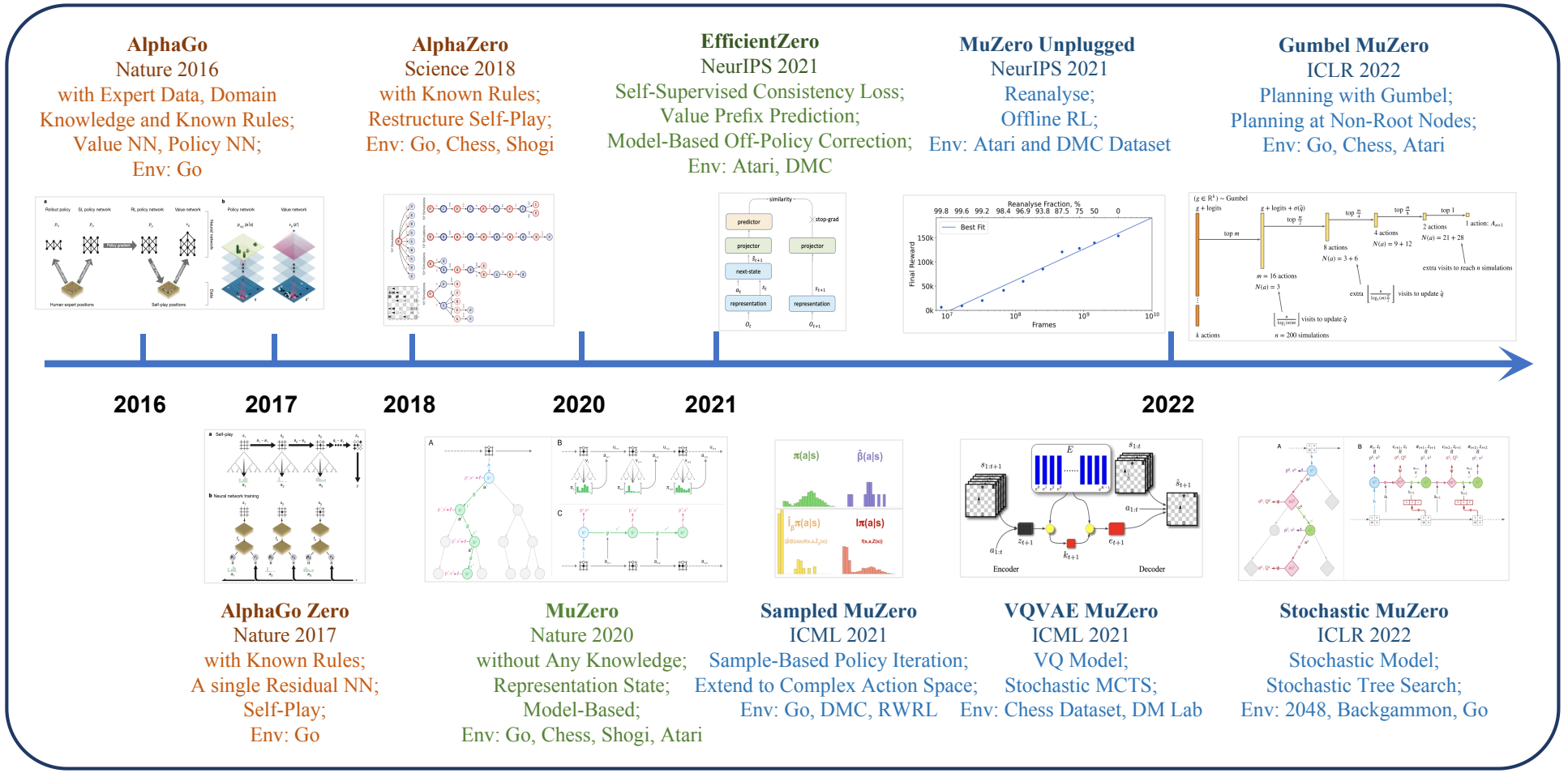
Оси — домены. В каждом результат прежнего SOTA принят за 1 (зелёный шестиугольник), у EfficientZero V2 и DreamerV3 — средний нормированный результат.

EfficientZero V2 превосходит прежний SOTA или близок к нему во всех доменах и лучше DreamerV3 в 50 из 66 задач.

Код: github.com/Shengjiewang-Jason/EfficientZeroV2

Wang et al. *EfficientZero V2: Mastering Discrete and Continuous Control with Limited Data*. ICML, 2024. arXiv:2403.00564.

Открытые реализации: LightZero



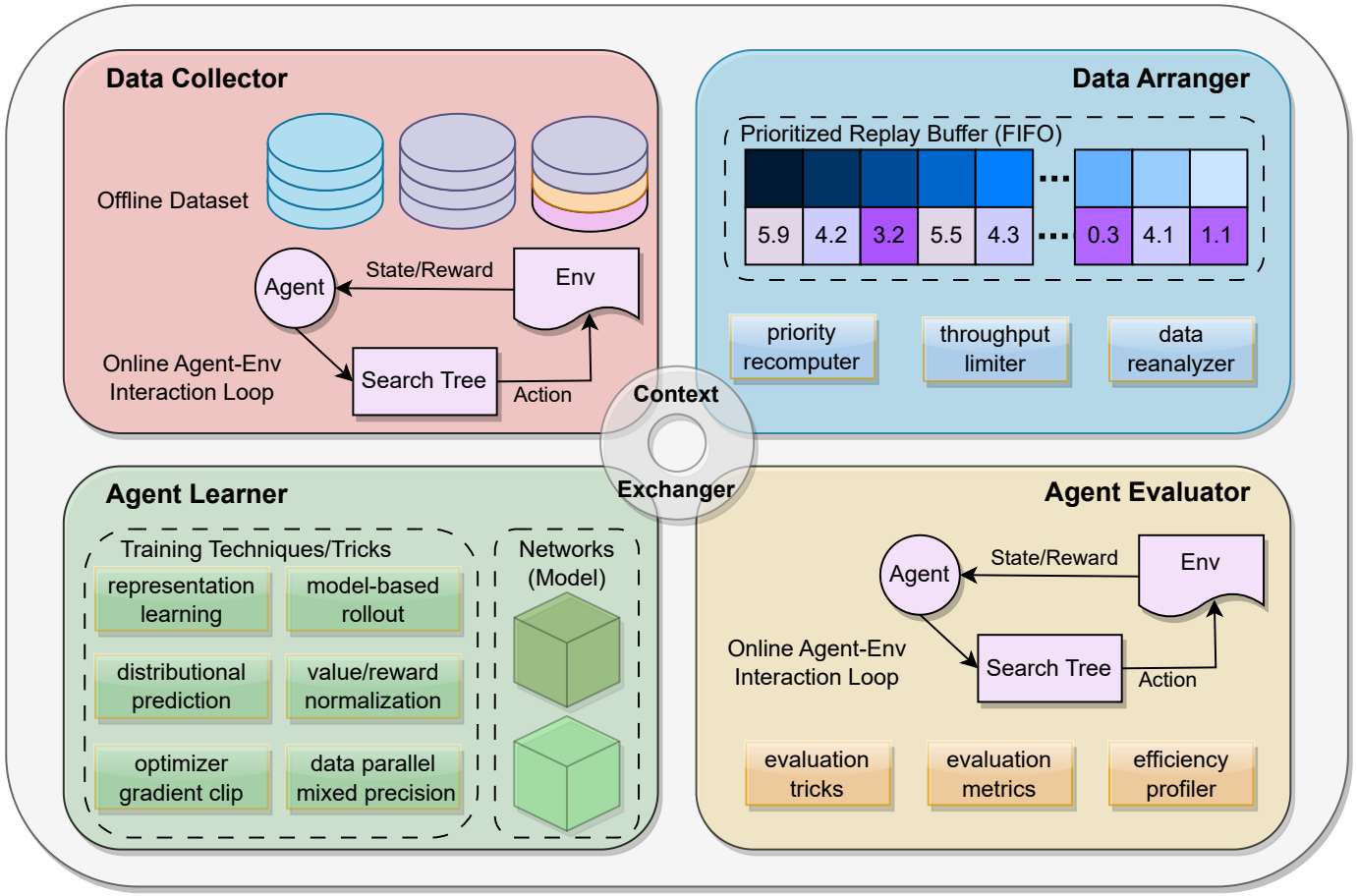
LightZero: A Unified Benchmark for Monte Carlo Tree Search in General Sequential Decision Scenarios

Y. Niu, Y. Pu, Z. Yang, X. Li, T. Zhou, J. Ren, S. Hu, H. Li, Y. Liu · NeurIPS 2023

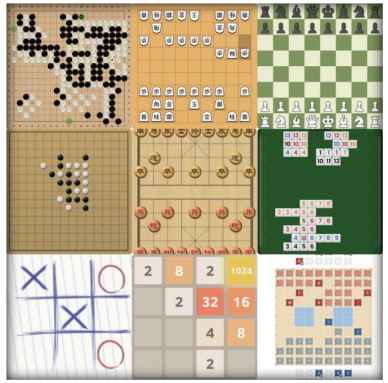
Код: github.com/opendilab/LightZero

Niu et al. *LightZero: A Unified Benchmark for Monte Carlo Tree Search in General Sequential Decision Scenarios*. NeurIPS Datasets and Benchmarks Track, 2023. arXiv:2310.08348.

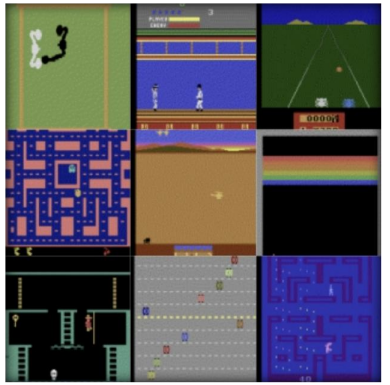
Открытые реализации: LightZero



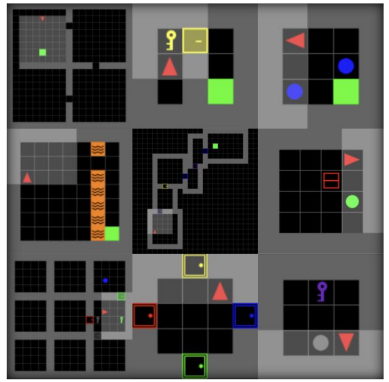
Настольные игры



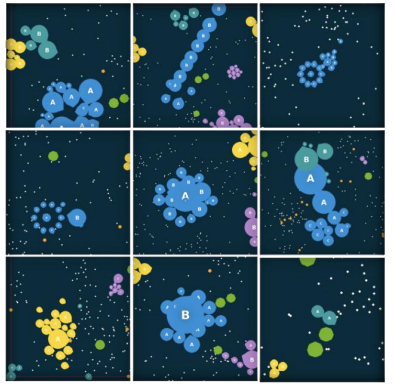
Atari



MiniGrid



GoBigger



Единый бенчмарк MCTS/MuZero: обучение разбито на 4 отдельных модуля
Код: github.com/opendilab/LightZero

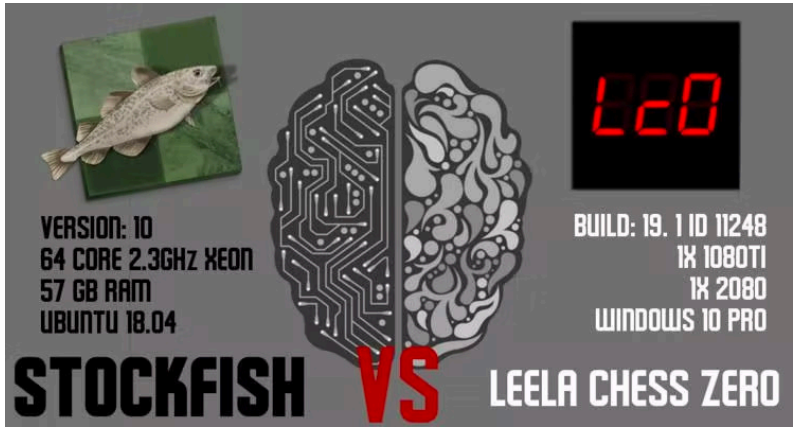
Niu et al. *LightZero: A Unified Benchmark for Monte Carlo Tree Search in General Sequential Decision Scenarios*. NeurIPS Datasets and Benchmarks Track, 2023. arXiv:2310.08348.

Открытые реализации: Lc0 (Leela Chess Zero)

Места Lc0 в Top Chess Engine Championship (TCEC)

Сезон	Дивизион					Инфрафинал	Суперфинал
	4	3	2	1	Премьер		
12 (2018)	8-е	—	—	—	—	—	—
13 (2018)	1-е	3-е	—	—	—	—	—
14 (2018)	—	1-е	1-е	1-е	2-е	—	**2-е**
15 (2019)	—	—	—	—	2-е	—	**1-е**
16 (2019)	—	—	—	—	**3-е**	—	—
17 (2020)	—	—	—	—	1-е	—	**1-е**
18 (2020)	—	—	—	—	2-е	—	**2-е**
19 (2020)	—	—	—	—	2-е	—	**2-е**
20 (2020)	—	—	—	—	1-е	—	**2-е**
21 (2021)	—	—	—	—	2-е	—	**2-е**
22 (2022)	—	—	—	—	3-е	**3-е**	—

Цвет — итоговое место Lc0 в сезоне; «—» — не играл на этом этапе.



Движок с открытым кодом (с 2018 г.) и проект добровольных распределённых вычислений на основе AlphaZero.

Как в AlphaZero, сеть выдаёт стратегию π_θ для поиска и оценку позиции V_ϕ; обучение — RL на партиях с самим собой, их играют компьютеры добровольцев.

Сопоставим по силе со Stockfish, ведущей классической программой; выиграл у него суперфиналы TCEC 15 и 17.

Код: github.com/LeelaChessZero/lc0

«Leela Chess Zero». Wikipedia (дата обращения: 10.10.2026), раздел «Results summary». en.wikipedia.org/wiki/Leela_Chess_Zero. Данные таблицы — архив TCEC: tcec-chess.com/archive.html.

05

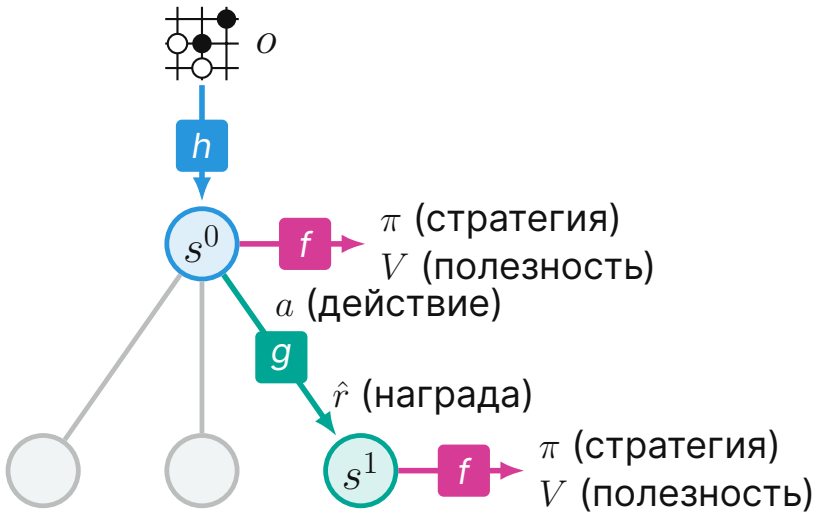
Dreamer



MuZero vs Dreamer

MuZero

MCTS по модели на каждом шаге

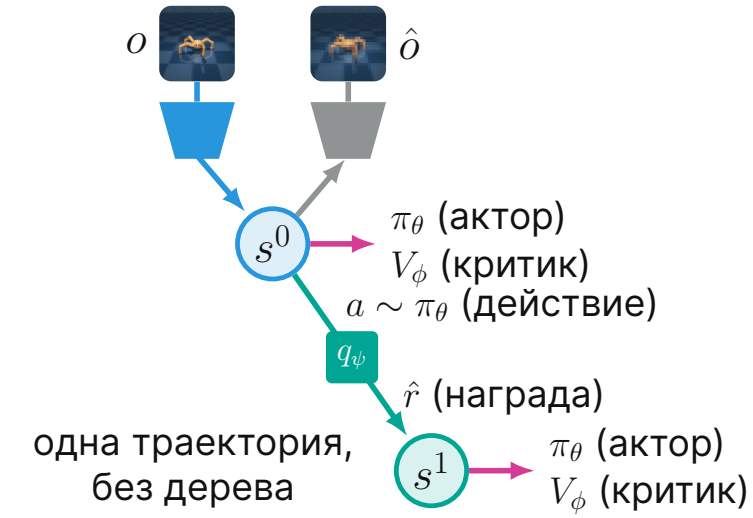


MuZero: 3 сети

предсказание	f:	$s \to \pi, V$
динамика	g:	$s, a \to \hat{r}, s$
представление	h:	$o \to s$

Dreamer

учится в воображении, действует без поиска

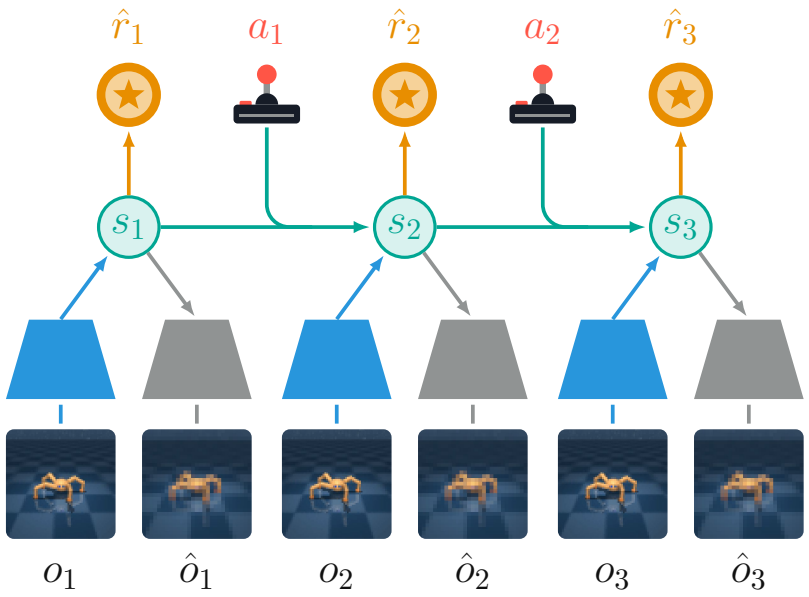


Dreamer: 5 сетей

актор	π_θ:	$s \to a$
критик	V_ϕ:	$s \to V$
динамика	q_ψ:	$s, a \to \hat{r}, s$
представление	p_ψ:	$o \to s$
декодер	q_ψ:	$s \to \hat{o}$

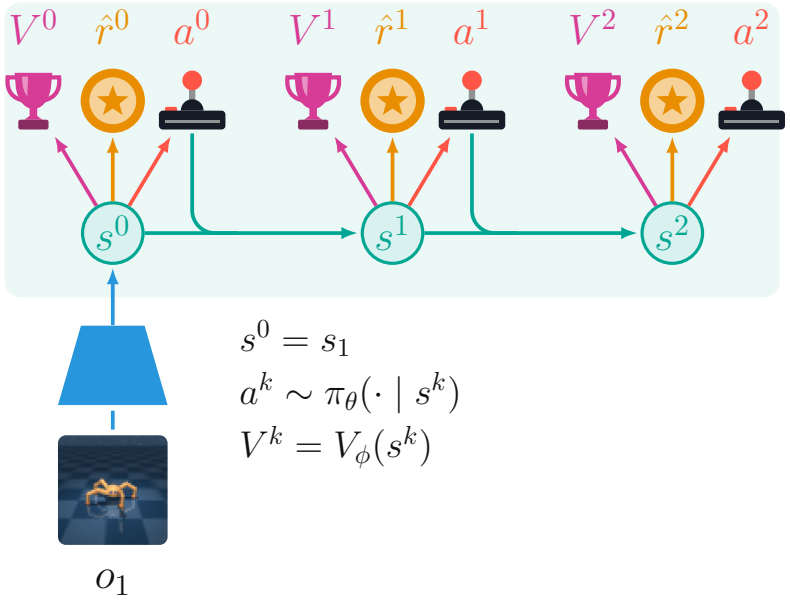
Dreamer

воображение: только модель, без наблюдений



Учим динамику среды
(по данным)

$\hat{o}_t \approx o_t,\ \hat{r}_t \approx r_t$ (обучаем ψ)



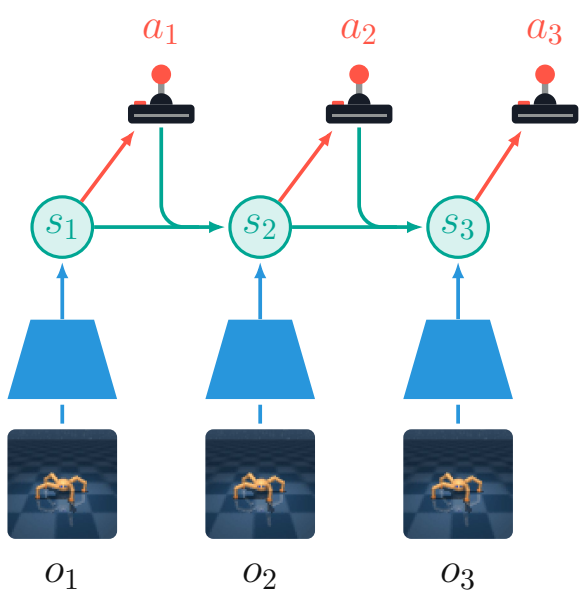
$s^0 = s_1$

$a^k \sim \pi_\theta(\cdot \mid s^k)$

$V^k = V_\phi(s^k)$

Учим поведение агента
(в воображении)

$\max G^\lambda\ (\theta),\ V^k \approx G^\lambda\ (\phi)$



Взаимодействие со средой
(сбор данных)

$a_t \sim \pi_\theta(\cdot \mid s_t)$, данные $\to \mathcal{D}$

Схема: рис. 3 из Hafner et al. *Dream to Control: Learning Behaviors by Latent Imagination*. ICLR, 2020. arXiv:1912.01603.

Псевдокод DreamerV1

Алгоритм 3. Dreamer

инициализировать $\mathcal{D}$ (S случайных эпизодов) и ψ, θ, ϕ
while нет сходимости **do**
  for $c \leftarrow 1, \dots, C$ **do** — C шагов обучения
    // Учим динамику среды (по данным)
    выбрать B отрезков длины L: $\{(a_{t-1}, o_t, r_t)\}_{t=t_0}^{t_0+L} \sim \mathcal{D}$
    вычислить состояния $s_t \sim p_\psi(s_t \mid s_{t-1}, a_{t-1}, o_t)$
    $\psi \leftarrow \psi - \alpha \nabla_\psi \mathcal{L}(\psi)$ (формула справа)
    // Учим поведение агента (в воображении)
    вообразить H шагов $\{(s^k, a^k)\}_{k=0}^{H}$ из каждого $s^0 = s_t$
    предсказать награды $\hat{r}^k = \mathbb{E}_{q_\psi}[r^k \mid s^k]$ и полезности $V_\phi(s^k)$
    вычислить λ-отдачи $G^\lambda(s^k)$ (формула справа)
    $\theta \leftarrow \theta + \alpha \nabla_\theta \sum_k G^\lambda(s^k)$
    $\phi \leftarrow \phi - \alpha \nabla_\phi \sum_k \frac{1}{2} \left\| V_\phi(s^k) - G^\lambda(s^k) \right\|^2$
  // Взаимодействие со средой (сбор данных)
  $o_0 \leftarrow$ `env.reset()`
  for $t \leftarrow 0, \dots, T-1$ **do**
    $s_t \sim p_\psi(s_t \mid s_{t-1}, a_{t-1}, o_t)$ — Восстанавливаем состояние по истории
    $a_t \sim \pi_\theta(a_t \mid s_t)$ — Выбираем действие
    добавить к a_t шум для исследования
    $r_{t+1}, o_{t+1} \leftarrow$ `env.step(`a_t`)`
  $\mathcal{D} \leftarrow \mathcal{D} \cup \{(o_0, a_0, r_1, o_1, \dots, a_{T-1}, r_T, o_T)\}$

Псевдокод: алгоритм 1 из Hafner et al. *Dream to Control: Learning Behaviors by Latent Imagination*. ICLR, 2020. arXiv:1912.01603.

Компоненты модели

Представление	$p_\psi(s_t \mid s_{t-1}, a_{t-1}, o_t)$
Функция перехода	$q_\psi(s_t \mid s_{t-1}, a_{t-1})$
Функция вознаграждения	$q_\psi(r_t \mid s_t)$
Декодер	$q_\psi(o_t \mid s_t)$
Актор	$\pi_\theta(a_t \mid s_t)$
Критик	$V_\phi(s_t)$

Функция потерь модели мира

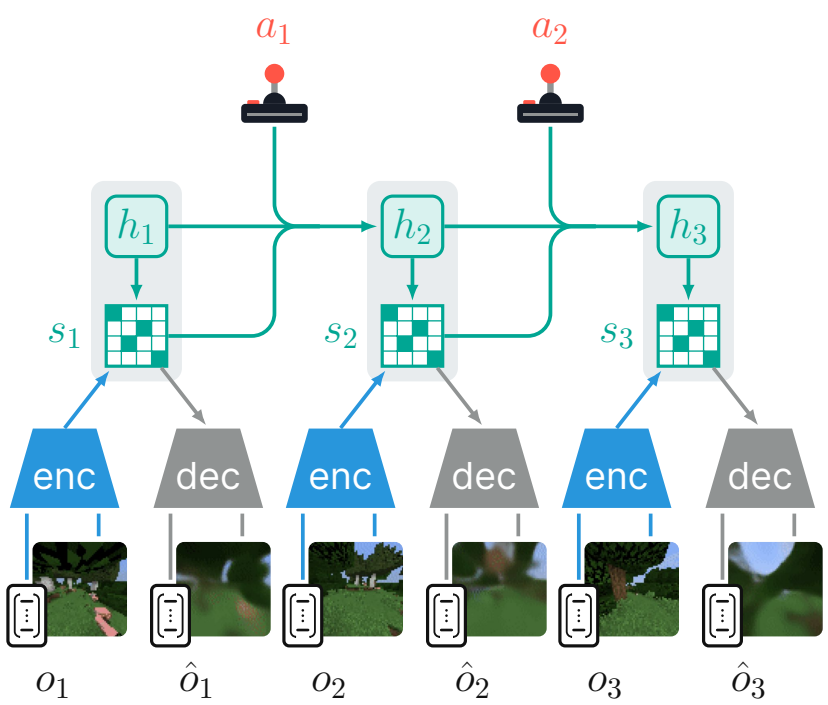
$$\mathcal{L}(\psi) = \sum_t \left[\ell_t^{\mathrm{o}} + \ell_t^{\mathrm{r}} + \ell_t^{\mathrm{KL}} \right]$$

$\ell_t^{\mathrm{o}} = -\ln q_\psi(o_t \mid s_t)$ — реконструкция
$\ell_t^{\mathrm{r}} = -\ln q_\psi(r_t \mid s_t)$ — награда
$\ell_t^{\mathrm{KL}} = \beta\, \mathrm{KL}\left(p_\psi \,\|\, q_\psi\right)$ — переход $\approx$ представление

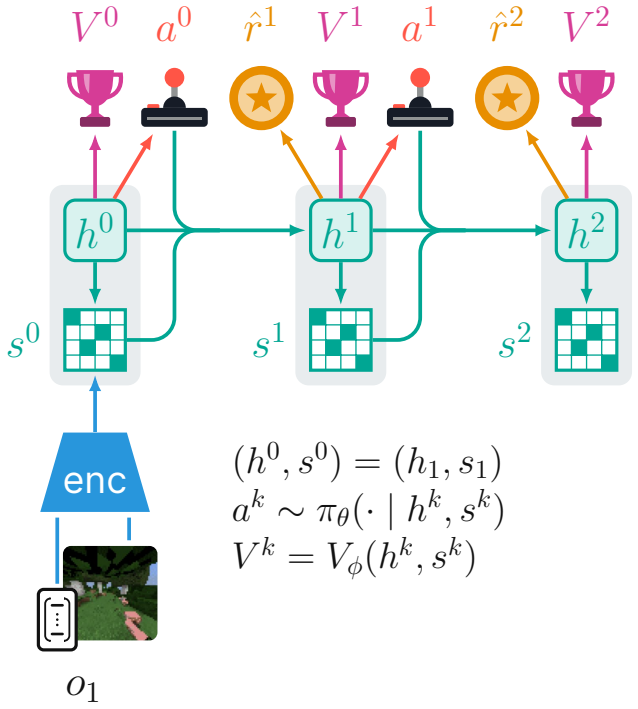
λ-отдача

$$G^\lambda(s^k) = \hat{r}^{k+1} + \gamma \left[(1-\lambda)\, V_\phi(s^{k+1}) + \lambda\, G^\lambda(s^{k+1}) \right]$$
$$G^\lambda(s^H) = V_\phi(s^H)$$

DreamerV3

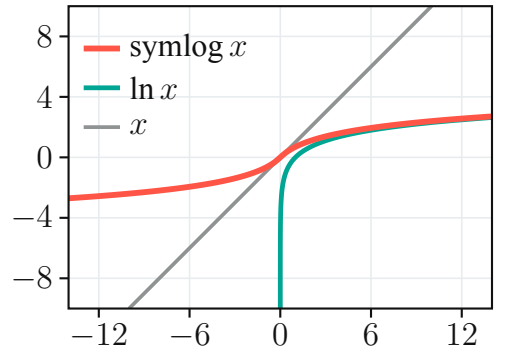


(a) Обучение модели мира



(b) Обучение актора и критика

$$\operatorname{symlog}(x) = \operatorname{sign}(x) \ln(|x| + 1)$$

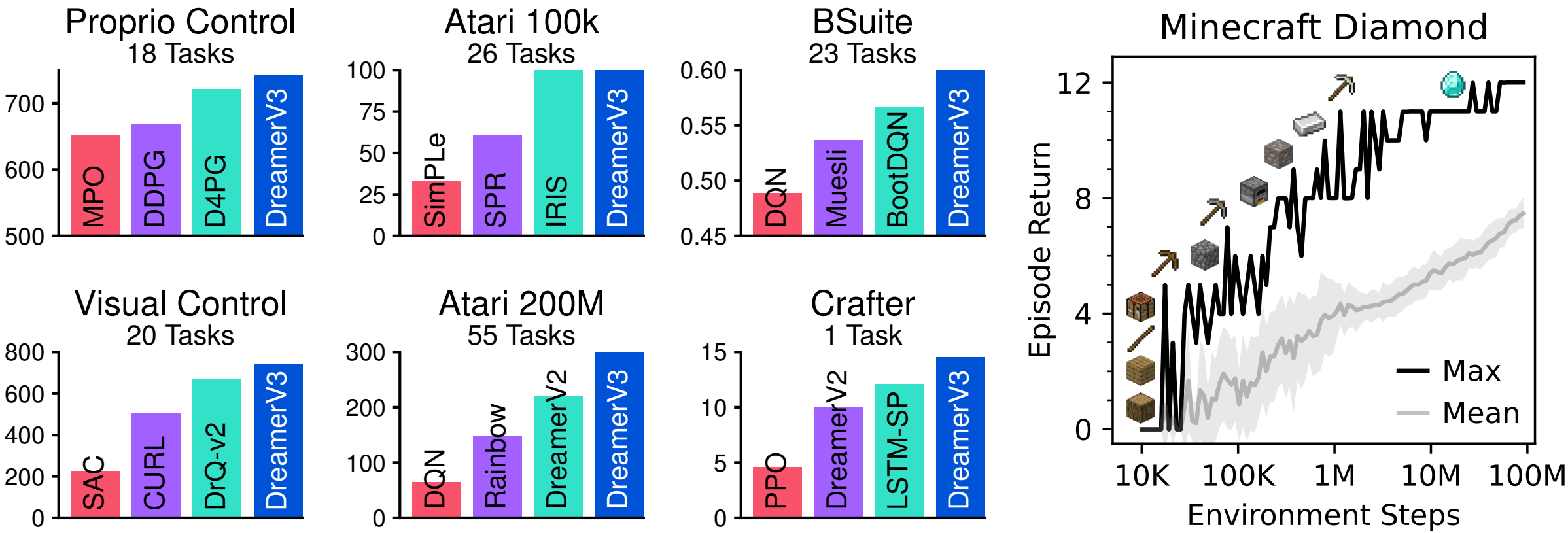


symlog сжимает большие по модулю значения и почти не меняет малые. Так DreamerV3 предсказывает входы, награды и полезности, и награды любого масштаба не нужно нормализовать.

Модель мира кодирует входы в дискретное представление s_t, предсказывает его по рекуррентному состоянию h_t и действиям a_t и учится, восстанавливая входы. Актор и критик обучаются на воображаемых латентных траекториях.

Схема: рис. 3, график: рис. 4 из Hafner et al. *Mastering Diverse Domains through World Models*. arXiv, 2023. arXiv:2301.04104v1. Журнальная версия: Nature, 2025.

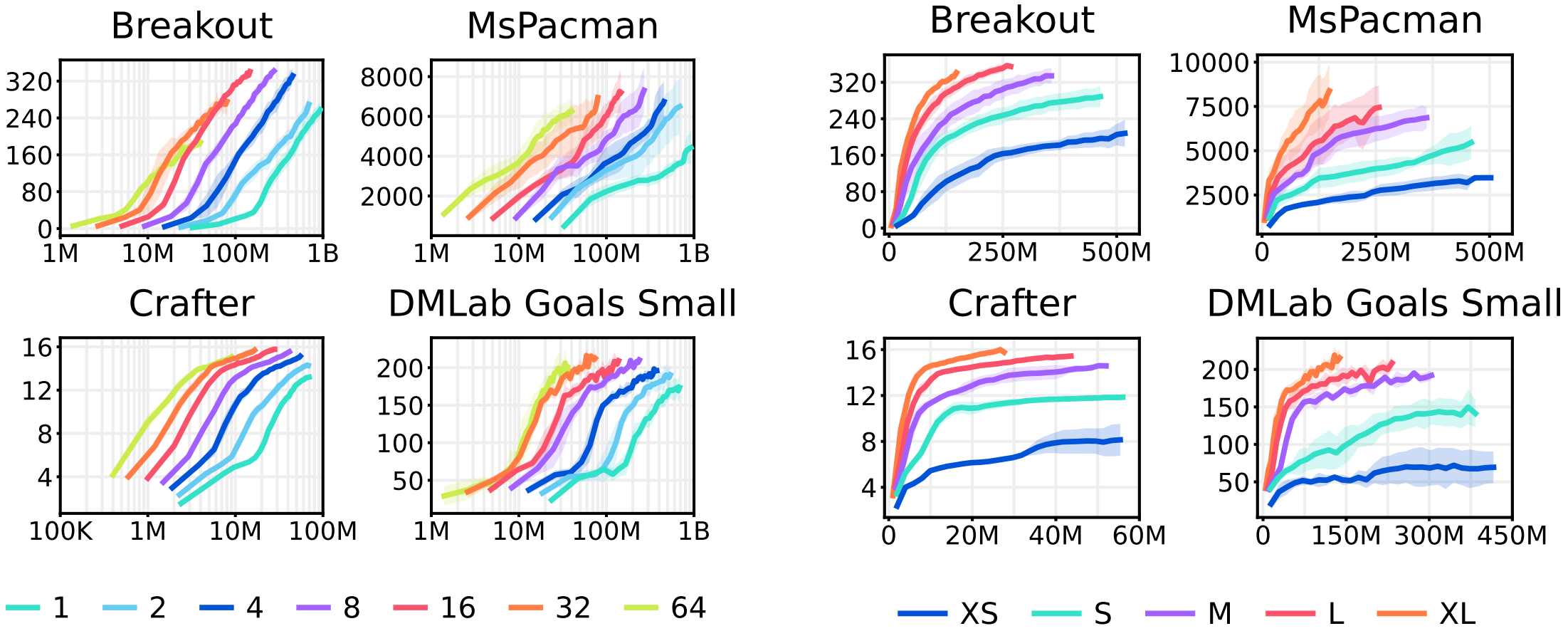
DreamerV3: результаты



Одни и те же гиперпараметры во всех доменах — DreamerV3 обходит специализированные алгоритмы.
Minecraft: первый алгоритм, добывший алмазы с нуля — без данных от людей и без curriculum.
Отдача: +1 за каждый из 12 предметов на пути к алмазу; Max и Mean — максимум и среднее по 40 запускам.

Hafner et al. *Mastering Diverse Domains through World Models*. arXiv, 2023. arXiv:2301.04104v1. Журнальная версия: Nature, 2025.

Увеличение модели



(a) Training ratio: шагов из буфера на шаг в среде

(b) Размер модели: от 8 до 200 млн параметров

По оси x — шаги в среде. Чем больше training ratio, тем меньше шагов нужно для того же результата. Чем больше модель, тем выше и итоговый результат, и сэмпл-эффективность.

Hafner et al. *Mastering Diverse Domains through World Models*. arXiv, 2023. arXiv:2301.04104v1. Журнальная версия: Nature, 2025.

Dreamer: итоги

Crafter



- Сэмпл-эффективный алгоритм:
  мало шагов в среде
- Низкая вычислительная эффективность:
  дорого в GPU-часах
- DreamerV3 применим почти к любой среде
  без доп. настройки
- Хороший выбор для медленных сред

Бюджет Crafter — 1 млн шагов среды; с ним в таблице лидеров Crafter те же Score у Rainbow, PPO и DreamerV2.

APPO — в ~500 раз больше шагов, но за сутки на одном GPU; Dreamer — наоборот: мало шагов, много GPU-часов.

Код: github.com/Tviskaron/crafter-appo-baseline

APPO: ~500 млн шагов среды, < 24 ч на одном GPU

Достижение	Rainbow	PPO	DreamerV2	APPO
Collect Coal	0.0	0.4	14.7	**96.6**
Collect Diamond	0.0	0.0	0.0	**22.6**
Collect Drink	24.0	30.3	80.0	**94.0**
Collect Iron	0.0	0.0	0.0	**83.1**
Collect Sapling	97.4	66.7	86.6	**98.9**
Collect Stone	0.2	3.0	42.7	**99.3**
Collect Wood	74.9	83.0	92.7	**99.9**
Defeat Skeleton	0.7	0.2	2.6	**89.3**
Defeat Zombie	39.6	2.0	53.1	**95.2**
Eat Cow	26.1	12.0	17.1	**93.3**
Eat Plant	0.0	0.0	0.1	**1.0**
Make Iron Pickaxe	0.0	0.0	0.0	**77.9**
Make Iron Sword	0.0	0.0	0.0	**55.4**
Make Stone Pickaxe	0.0	0.0	0.2	**97.6**
Make Stone Sword	0.0	0.0	0.3	**98.5**
Make Wood Pickaxe	4.8	21.1	59.6	**99.7**
Make Wood Sword	9.8	20.1	40.2	**99.0**
Place Furnace	0.0	0.1	1.8	**98.0**
Place Plant	94.2	65.0	84.4	**99.0**
Place Stone	0.0	1.7	29.0	**99.0**
Place Table	52.3	66.1	85.7	**99.0**
Wake Up	93.3	92.5	92.8	**97.0**
Score	4.3	4.6	10.0	**50.0**

Доля эпизодов с достижением, %;
Score — среднее геометрическое этих долей

Вопросы?